

PROJECT APEX — Institutional-Grade Quant Trading System

The Complete Master Plan & Build Blueprint (End-to-End Guide)

Personal Quant Research Program

Version 1.0 — 2026

DOCUMENT CONTROL

Field	Value
Document title	Project APEX — Quant Trading System Master Plan
Version	1.0
Status	Approved for execution
Owner	You (System Architect & Operator)
Purpose	Single source of truth for designing, building, testing, and operating a complete quantitative trading system at zero budget
Scope	Research, backtesting, strategy mining, validation, portfolio & risk management, execution support, monitoring, and continuous improvement
Out of scope	Paid data feeds, paid APIs, cloud compute, high-frequency trading, options market-making

Revision history

Version	Date	Change
0.1	2026	Initial outline and module map
0.5	2026	Added strategy miner, validation lab, QA gates
1.0	2026	Complete master plan with acceptance criteria and runbooks

How to use this document. This is a working blueprint, not reading material. Each module chapter ends with an **Acceptance & Quality Inspection Checklist** — do not consider a module “done” until every box in its checklist passes with evidence (a query result, a test run, a

screenshot, a log). Keep this document open while building; when reality diverges from the plan, update the document (it is version-controlled with the code).

MASTER TABLE OF CONTENTS

PART A — FOUNDATIONS 1. Executive Summary 2. Vision, Objectives & Success Criteria 3. Honest Constraints & Realistic Expectations 4. Market Selection Analysis (Crypto vs Forex vs Equities) 5. The Twelve Core Design Principles

PART B — SYSTEM ARCHITECTURE 6. High-Level Architecture (The 12 Layers) 7. Technology Stack — Repo Combination Strategy 8. Project Structure, Environment & Configuration

PART C — MODULE SPECIFICATIONS 9. Module 1 — Data Lake 10. Module 2 — Feature & Regime Factory 11. Module 3 — Strategy Miner (Brute-Force, Genetic, ML) 12. Module 4 — Backtest & Simulation Engine 13. Module 5 — Trade Database (The Edge Mine) 14. Module 6 — Edge Analytics Engine 15. Module 7 — Validation Lab (Anti-Overfitting Suite) 16. Module 8 — Portfolio Construction Engine 17. Module 9 — Risk Engine 18. Module 10 — Command Center (UI) 19. Module 11 — Execution & Live Loop 20. Module 12 — Monitoring, Alerts & Edge-Decay Detection

PART C-2 — BUILDER’S COOKBOOK (DETAILED REFERENCES) C2.1 Data Dictionary & Feature Catalog C2.2 Strategy Building-Block Catalog C2.3 Starter Strategy Playbook (8 Fully Specified Strategies) C2.4 Worked Example — End-to-End Walkthrough C2.5 Module Interface Contracts C2.6 Testing Cookbook C2.7 Operational Playbooks & Incident Response C2.8 Report Templates (Daily / Weekly / Monthly) C2.9 Journal & Edge Card Templates C2.10 The 90-Day Condensed Quick-Start C2.11 Code Skeletons for Core Components C2.12 Module Build Task Sequences (Step-by-Step)

PART D — DATA & MATH REFERENCE 21. Complete Database Schemas 22. Formula & Metrics Library 23. Forex Plug-In Specification (Future Market)

PART E — IMPLEMENTATION PLAN 24. Phased Build Plan (Week-by-Week) 25. Engineering Standards (Git, Testing, Review, Docs) 26. Quality Assurance & Acceptance Test Plan

PART F — OPERATIONS 27. Daily / Weekly / Monthly Runbooks 28. Strategy Lifecycle Management 29. Manual Execution & Psychology Protocol 30. Capital Plan, Costs & Risk Disclosures

PART G — FUTURE & APPENDICES 31. Future Enhancements Roadmap 32. Scaling Path & Hardware Notes A. Glossary B. Repository & Resource Directory C. SQL Query Library D. Reading List E. Master Project Sign-Off Checklist F. Common Pitfalls & Troubleshooting Guide G. Worked Mathematics & Sizing Workbook H. Failure-Mode Library (25 Ways Quant Systems Die) I. Complete Configuration Reference J. Extended SQL Query Library K. Compounding, Drawdown & Risk-of-Ruin Mathematics L. Data Quality Field Guide M. Builder’s Weekly Schedule Templates N. Expanded Glossary of Quant Terms O. Pre-Live Simulation Drill Pack P. Frequently-Used Reference Tables

PART H — TRADER’S REFERENCE LIBRARY 33. Statistics Primer for Traders 34. Crypto Market Microstructure Primer 35. KPI Definitions & Dashboard Metrics 36. First-Year Operating Calendar 37. The Learning Track (Study Plan Alongside the Build) 38. Self-Interrogation FAQ 39. Universe & Correlation Management 40. QA Quick-Reference Cards (Printable) 41. Case Studies — Anatomy of a Failure & of a Success 42. The First 100 Live Trades — What to Expect 43. Document & Template Library 44. Year-2 Detailed Roadmap (Months 13–24) 45. The Daily Boot Sequence & Operating Manifesto 46. Master Index of Gates, Checklists & Controls

PART A — FOUNDATIONS

1. Executive Summary

Project APEX is a complete, zero-budget quantitative trading system that you will build and operate alone. Its purpose is not to “find a winning strategy” — it is to build **a factory that continuously discovers, validates, filters, sizes, monitors, and retires strategies**, the same loop every professional quant firm runs, rebuilt from free open-source components.

The system rests on five pillars:

1. **A data lake** of free, institutional-quality crypto market data (Binance public archive: tick-to-daily history since 2017, unlimited downloads, no account required), stored locally in Parquet + DuckDB.
2. **A strategy miner** that automatically discovers candidate strategies — first by brute-force combinatorial search over indicator/filter/exit building blocks, later by genetic programming that evolves strategy logic itself, and finally by local machine learning (no paid AI APIs anywhere).
3. **A trade database (“the edge mine”)** in which every trade from every backtest and every live session is stored with ~30 labels (strategy, parameters, pair, session, hour, day, volatility regime, trend regime, exit reason, MFE/MAE...). This database is the system’s most valuable permanent asset: it lets you mine *when and where* each strategy works — your original edge idea, implemented with statistical rigor.
4. **A validation lab** that kills weak strategies before they cost money: walk-forward analysis, four kinds of Monte Carlo simulation, Probability of Backtest Overfitting (PBO), Deflated Sharpe Ratio, and regime stress tests. Expect to reject 95% of what the miner finds. That rejection rate is the system working correctly.
5. **A risk-first portfolio engine and command center**: volatility-targeted position sizing, HRP/CVaR allocation across strategies, hard circuit breakers, and a Streamlit dashboard that hands you a clear “edge card” each session so manual execution becomes checklist execution.

What success looks like (12–18 months): a system holding 100,000+ labeled trades; 5–10 validated strategies with out-of-sample-confirmed session/day/regime filters; a portfolio-level risk engine with circuit breakers you actually obey; a live track record of 100+ real trades whose expectancy matches the backtest within tolerance; and a monthly re-mining loop that keeps the edge fresh as markets evolve.

Build time to first validated paper trade: ~12 weeks. To first live trade: ~5–6 months.
Budget: ₹0 / \$0.

2. Vision, Objectives & Success Criteria

2.1 Vision statement

Build a personal quant research operation whose *process* is institutional-grade even though its *capital* is retail-sized — and let disciplined process, not luck, compound both skill and capital over time.

2.2 Objectives (SMART)

#	Objective	Measure	Target
01	Local data lake, self-updating	Pairs × years × timeframes coverage	20 pairs, 2017→now, 1m–1d
02	Automated strategy discovery	Miner variants evaluated per run	≥ 100,000 variants/run
03	Complete trade memory	Labeled trades in DuckDB	≥ 100,000 by month 6
04	Validation discipline	% of mined strategies rejected	≥ 90% rejected (healthy)
05	Portfolio of uncorrelated edges	Live-validated strategies	5–10
06	Risk control	Max portfolio drawdown	Hard stop at –20%
07	Live-vs-backtest fidelity	Live expectancy within 1.5σ of backtest	Yes, per strategy
08	Positive expectancy	Net expectancy after all costs, 6-month rolling	> 0 per active strategy
09	Operator discipline	Checklist adherence score (runbook §27)	≥ 95%
010	Zero recurring cost	Monthly spend on data/APIs/compute	\$0

2.3 What this system is NOT

- Not a get-rich-quick bot. It is a research and decision-support machine that makes *you* a disciplined, evidence-driven trader.
- Not HFT. We compete in timeframes (minutes to days) where speed doesn't decide outcomes.
- Not an AI magic box. Every “intelligent” component runs locally and free; statistics does the heavy lifting.

- Not a one-time project. It is an operating system for your trading career, maintained weekly.

3. Honest Constraints & Realistic Expectations

This chapter exists so that month-6-you doesn't quit out of misplaced expectations. Read it monthly.

3.1 You cannot beat institutions at their game. Funds have colocated servers, tick-level order flow, teams of PhDs, and billion-dollar data budgets. You will not out-speed or out-data them.

3.2 You can win the game they refuse to play. Institutional capital cannot bother with edges worth 0.3–1% per week on \$5,000 — it's meaningless to them and often too small or capacity-limited to trade. Retail-sized, patient, well-filtered edges in liquid-but-slow timeframes are exactly the niche this system targets.

3.3 Filtering turns break-even into profitable — it does not resurrect garbage.

Day/hour/session/regime filtering (your core idea) reliably improves mediocre strategies; it rarely saves a strategy with no underlying edge. The miner's job is to find raw material; the filter's job is to refine it.

3.4 Most discovered “edges” are noise. If you test 100,000 variants, thousands look brilliant by pure chance. This is the multiple-comparisons problem, and it is why Part C's Validation Lab (PBO, Deflated Sharpe, walk-forward) is non-negotiable. The uncomfortable truth: **the better the backtest looks, the more skeptical you must be.**

3.5 Edges decay. Market regimes shift; other participants arbitrage patterns away. Assume every edge has a half-life of 6–24 months. The monthly re-mining and edge-decay detector exist for this reason.

3.6 The operator is the biggest risk. Manual execution, revenge trading after losses, overriding the system “just this once,” stopping during drawdowns — operator behavior kills more retail systems than bad math. Chapter 29 is as important as any code.

3.7 Expectancy arithmetic (memorize):

Expectancy = (win_rate × avg_win) - (loss_rate × avg_loss) - costs
 A strategy is tradeable only if Expectancy > 0 AFTER realistic fees + slippage + spread, AND n ≥ 100 000 trades, AND it survives the Validation Lab.

3.8 Timeline honesty. Month 1–3: building, zero trading income. Month 4–6: paper trading, zero income. Month 6–12: small live trading, modest income if everything works. Compounding is slow at first and then not. Anyone promising faster is selling something.

4. Market Selection Analysis

4.1 The decision criteria

Criterion	Weight	Why it matters
Data cost & limits	25%	\$0 budget is absolute
Data quality & depth	20%	Backtest realism depends on it
History length	10%	More regimes = better validation
Market hours	10%	24/7 = 4.5× more data/year than 5-day markets
Tradability at tiny capital	15%	Live validation must be affordable
Paper-trading availability	10%	Free testnet/demo
Strategy diversity fit	10%	Trend + mean-reversion both present

4.2 Candidate comparison

Factor	Crypto (Binance)	Forex (Dukascopy)	US Equities	Indian Equities
Historical data cost	Free, unlimited bulk ZIPs	Free but heavily rate-limited	Intraday paywalled (Polygon etc.)	Needs funded broker account
Granularity available	Tick, aggTrades, 1s–1M klines	True tick w/ bid-ask	Daily free, intraday limited	Limited intraday depth
History start	2017 (BTC), 2018–2020 (alts)	2003+	Decades (daily only)	Varies
Market hours	24/7/365	24/5	6.5h × 5d	6.25h × 5d
Min live position	~\$5	~\$100+ (micro lots)	~\$50+	~₹500+
Free paper trading	Binance testnet (excellent)	MT5 demo (good)	Alpaca paper (good)	Limited
Spread/cost model	Real exchange spreads	Variable, session-dependent	Commissions + spread	STT + brokerage + slippage
Timezone complexity	None (pure UTC)	High (DST, sessions)	Medium	Low
Verdict	PRIMARY	Secondary (Phase 3 plug-in)	Later	Later

4.3 Decision

Primary market: Crypto, via Binance public data archive (data.binance.vision) + CCXT for live updates.

Rationale: (1) truly unlimited free downloads — no API key, no rate-limit gymnastics, tick-level history back to 2017; (2) 24/7 operation yields $\sim 4.5\times$ more bars per year than forex, which directly multiplies the statistical power of your hour/day edge mining; (3) single clean UTC timeline — no DST bugs corrupting session labels; (4) free testnet for paper trading and \$5 minimum live positions for cheap live validation; (5) both trend and mean-reversion regimes exist across 20+ liquid pairs.

Architecture consequence: all data flows through ONE abstract DataSource interface. Forex (Dukascopy) plugs in later as a second implementation (spec in Chapter 23) without touching any downstream module.

Universe (initial): BTCUSDT, ETHUSDT, BNBUSDT, SOLUSDT, XRPUSDT, ADAUSDT, DOGEUSDT, AVAXUSDT, LINKUSDT, MATICUSDT, LTCUSDT, DOTUSDT, ATOMUSDT, NEARUSDT, ARBUSDT, OPUSDT, INJUSDT, SUIUSDT, APTUSDT, FILUSDT — all USDT-margined perpetual/spot pairs with history since ≥ 2020 preferred.

5. The Twelve Core Design Principles

These principles govern every build decision. When in doubt, the principle that says “be more conservative” wins.

1. **The database is the product.** Strategies are disposable; the labeled trade database compounds in value forever. Never run a backtest whose trades aren’t logged.
2. **Costs first, always.** Every backtest models fees, spread, and slippage from day one. A backtest without costs is fiction.
3. **In-sample is for searching, out-of-sample is for deciding.** The miner never sees the test period. No exceptions, ever.
4. **Sample size or it didn’t happen.** No slice, filter, or strategy is trusted below $n=100$ trades (50 = provisional only).
5. **Simple beats clever.** A robust simple strategy beats a fragile complex one. Complexity must earn its place with OOS performance.
6. **Plateaus, not peaks.** Choose parameter regions where neighbors also work; a lone peak is an overfit artifact.
7. **Kill ruthlessly.** Rejecting 90–95% of candidates is the sign of a healthy pipeline, not a failure.
8. **Everything is reproducible.** Every run is config-driven, seeded, logged, and re-runnable byte-for-byte.
9. **Local-first, zero-cost.** If a component needs a paid API, it doesn’t belong in the core system.
10. **The system advises; discipline executes.** Until auto-execution is earned, the human executes the checklist — not impulses.

11. **Risk is a budget, not an afterthought.** Position size, exposure, and circuit breakers are computed by the Risk Engine, never by mood.
12. **Document as you build.** A module without a completed QA checklist and a doc page is not “done.”

PART B — SYSTEM ARCHITECTURE

6. High-Level Architecture — The 12 Layers

L1	DATA LAKE	Binance bulk archive → Parquet → DuckDB (klines 1m→1d, aggTrades, funding, calendar)
L2	FEATURE FACTORY	indicators, regimes, session/hour labels, volatility & trend states → features.parquet
L3	STRATEGY MINER	Miner-1 brute force → Miner-2 genetic → Miner-3 local ML; outputs candidate configs
L4	BACKTEST ENGINE	vectorbt (sweeps) + event-driven engine (NautilusTrader) for realistic validation
L5	TRADE DATABASE	★ every trade from every run, 30+ labels ★ (DuckDB: trades, runs, signals, equity)
L6	EDGE ANALYTICS	SQL slicing by session/day/hour/regime + significance testing → edge cards
L7	VALIDATION LAB	walk-forward, Monte Carlo ×4, PBO, Deflated Sharpe, regime stress, kill list

L8	PORTFOLIO ENGINE	skfolio: HRP / CVaR risk parity allocation across strategies & pairs; correlation guard
L9	RISK ENGINE	vol targeting, VaR/CVaR (arch/GARCH), position sizing, circuit breakers
L10	COMMAND CENTER	Streamlit UI: overview, miner control, edge explorer, validation, portfolio, journal, data manager
L11	EXECUTION LOOP	edge cards → alerts (Telegram) → manual / testnet execution → live trades logged to L5
L12	MONITORING	live-vs-backtest drift, edge-decay detector, weekly reports, monthly re-mine scheduler

6.1 Data flow (the daily life of the system)

1. **Nightly (02:00):** Data Lake pulls the previous day's candles for all pairs → validates → appends to Parquet → rebuilds DuckDB views.
2. **Nightly (02:30):** Feature Factory recomputes indicators/regimes for the new bars.
3. **On demand / weekly:** Miner launches search runs over the in-sample period; survivors are handed to the Backtest Engine for realistic replay.
4. **Every run:** all trades → Trade Database.
5. **Nightly (03:00):** Edge Analytics recomputes slice statistics; Edge-Decay detector compares live vs historical expectancy per strategy.
6. **Each session (before you trade):** Command Center shows today's Edge Cards + risk budget; Telegram alert fires when a valid setup window opens.
7. **You execute** (manually at first); the fill is journaled into the Trade Database with source='live'.

8. **Monthly:** Walk-forward re-validation on all data including new live trades; Portfolio Engine rebalances strategy weights; miner re-runs on refreshed data.

6.2 The one-way research wall (critical architecture rule)

|←— IN-SAMPLE (2017-2022): miner + analytics may touch —→|||←—
OUT-OF-SAMPLE (2023→now): validation only →|

Code enforcement: the Data Lake exposes `get_data(start, end)`; the Miner process is *only ever passed* `end='2022-12-31'`. The OOS segment is loaded only inside the Validation Lab. This is enforced by configuration, not by memory.

7. Technology Stack — Repo Combination Strategy

7.1 Selection criteria applied to every repo

- Actively maintained (commits within last 6 months)
- Permissive license (MIT/BSD/Apache)
- Local & free at our usage level
- Composable (library, not just an app)
- Community size (issues get answered)

7.2 The chosen stack

Layer	Component	Repo / Package	License	Role
Data	Binance archive client	Custom downloader (~200 lines, stdlib requests + zipfile)	MIT	Bulk history ingestion
Data	CCXT	ccxt	MIT	Live/incremental updates, exchange abstraction
Storage	Parquet	pyarrow	Apache	Columnar data files
Storage	DuckDB	duckdb	MIT	Analytical SQL over Parquet; trade DB
Compute	pandas / numpy	pandas, numpy	BSD	Core dataframes
Indicators	pandas-ta	pandas-ta	MIT	130+ indicators
Sweeps	vectorbt (OSS)	vectorbt	Apache	Massive parameter grids, fast screening
Realistic backtest	NautilusTrader	nautilus_trader	LGPL	Event-driven fills, fees,

Layer	Component	Repo / Package	License	Role
				slippage; backtest≈live parity
Quick backtest	backtesting.py	backtesting	AGPL (dev tool only)	Rapid single-idea prototyping
Reports	quantstats	quantstats	Apache	Tear sheets, metrics, heatmaps
Factor analysis	alphalens-reloaded	alphalens-reloaded	Apache	Signal IC analysis
Portfolio	skfolio	skfolio	BSD	HRP, risk parity, CVaR opt, walk-forward CV
Alt portfolio	Riskfolio-Lib	riskfolio-lib	BSD	Drawdown-constrained allocation (comparison)
Volatility	arch	arch	NCSA	GARCH, VaR forecasting
Stats	statsmodels / scipy	—	BSD	Tests, cointegration, distributions
Genetic mining	DEAP + gplearn	deap, gplearn	LGPL / BSD	Genetic programming engine (Miner-2)
Local ML	scikit-learn / XGBoost / LightGBM	—	BSD	Miner-3, trade-success classifier
Feature extraction	tsfresh	tsfresh	MIT	Automated time-series features
UI	Streamlit	streamlit	Apache	Command Center
Plotting	plotly / matplotlib	—	MIT	Interactive charts
Alerts	python-telegram-bot	—	LGPL	Telegram notifications
Scheduling	APScheduler	apscheduler	MIT	Nightly jobs, alert windows
Live trading	Freqtrade	freqtrade	GPL	Battle-tested

Layer	Component	Repo / Package	License	Role
(later)				crypto execution + FreqAI local ML
Notebook	Jupyter	—	BSD	Exploratory research
DB UI	DuckDB CLI / Harlequin	—	MIT	Ad-hoc SQL

7.3 Deliberately excluded (and why)

Excluded	Reason
Backtrader	Unmaintained since 2023; creator departed; bugs unpatched
Zipline / zipline-reloaded	Extremely slow; equity-centric; heavy dependency chain
PyAlgoTrade, Catalyst, Blankly, Bulbea Freqtrade (as core)	Abandoned Excellent for crypto live trading, but its backtester is strategy-singular — poor for mass mining; we adopt it at the execution layer only
Qlib, FinRL	ML research frameworks; not production trading systems; dependency hell
Jesse, Hummingbot	Crypto-specific apps; Hummingbot targets market-making — revisit for that module later
Paid vectorbt PRO / StrategyQuant / Trade Ideas	Budget = 0; OSS covers our needs
LLM APIs in the trading loop	Recurring cost; no measurable edge vs. local statistics

7.4 Combining repos without creating a monster (integration rules)

1. **Repos are libraries, never frameworks-we-live-inside.** Our own thin orchestration layer (`/src`) calls them; we never fork them.
2. **One data contract.** Everything speaks pandas DataFrames (OHLCV indexed by UTC timestamp) and our DuckDB schemas. Adapters translate at the boundary.
3. **Pin every version** in `requirements.txt`; upgrade deliberately, one at a time, with the test suite green before and after.
4. **Wrap external calls** in our own functions (`run_vectorbt_sweep()`, `run_nautilus_backtest()`) so a future swap of engines touches one file.

8. Project Structure, Environment & Configuration

8.1 Repository layout

```
apex/
├── README.md                ← project overview + how to run
├── requirements.txt          ← pinned versions
├── pyproject.toml
├── config/
│   ├── system.yaml          ← global config (paths, dates, seeds)
│   ├── universe.yaml        ← pairs, timeframes, history depth
│   ├── risk.yaml            ← all risk limits & sizing params
│   └── strategies/          ← one YAML per strategy/candidate
├── data/                    ← DATA LAKE (Parquet; gitignored)
│   ├── raw/binance/<pair>/<tf>.parquet
│   ├── features/<pair>/<tf>.features.parquet
│   └── calendar/events.parquet
├── db/
│   └── apex.duckdb           ← trade database + views
├── notebooks/               ← exploratory research (Jupyter)
├── src/
│   └── datalake/             ← L1 downloaders, validators,
resamplers
│   ├── features/            ← L2 indicators, regimes, labels
│   ├── miner/               ← L3 brute force / genetic / ML miners
│   ├── backtest/            ← L4 vectorbt + nautilus wrappers
│   ├── tradesdb/            ← L5 schema, writers, views
│   ├── edge/                ← L6 slicing, significance, edge cards
│   ├── validation/          ← L7 walk-forward, montecarlo, pbo, dsr
│   ├── portfolio/           ← L8 allocation (skfolio wrappers)
│   ├── risk/                ← L9 sizing, VaR, circuit breakers
│   ├── ui/                  ← L10 Streamlit app (pages/)
│   ├── execution/           ← L11 alerts, journal, testnet bridge
│   └── monitoring/          ← L12 decay detector, reports
├── tests/                   ← pytest suite (unit + integration)
├── runs/                    ← every backtest/mining run (config +
logs + metrics)
├── docs/                    ← this master plan + module docs +
decisions log
```

8.2 Environment setup

1. Python 3.11+ recommended

```
python -m venv .venv && source .venv/bin/activate
```

```
# 2. Core install (pinned list maintained in requirements.txt)
pip install pandas numpy pyarrow duckdb pandas-ta vectorbt \
    backtesting quantstats alphasens-reloaded \
    scikit-learn xgboost lightgbm statsmodels scipy arch \
    skfolio riskfolio-lib ccxt streamlit plotly \
    apscheduler python-telegram-bot deap gplearn tsfresh \
    pytest pytest-cov pyyaml tqdm rich
```

```
# 3. NautilusTrader (heavier; install when reaching Module 4)
pip install nautilus_trader
```

```
# 4. Sanity
python -c "import duckdb, vectorbt, skfolio; print('stack ok')"
```

8.3 Global configuration contract

```
# config/system.yaml
paths:
  data: data/
  db: db/apex.duckdb
  runs: runs/
time:
  insample_start: "2017-08-17"
  insample_end:   "2022-12-31" # ← the research wall
  oos_start:      "2023-01-01"
random_seed: 42 # everything reproducible
costs:
  taker_fee_bps: 5 # 0.05% per side (Binance spot tier)
  slippage_bps: 2 # base assumption; stress at 2-3x
validation:
  min_trades_slice: 100
  min_trades_provisional: 50
  mc_sims: 10000
  pbo_threshold: 0.20
```

Rule: no magic numbers in code. Every threshold lives in YAML and is recorded into each run's metadata, so any historical result can be traced to the exact configuration that produced it.

PART C — MODULE SPECIFICATIONS

Each module chapter follows the same contract: Purpose → Responsibilities → Inputs/Outputs → Detailed Design → Implementation Steps → Configuration → Edge Cases → Acceptance & Quality Inspection Checklist. A module is “complete” only when every checklist item passes with evidence.

9. Module 1 — Data Lake

9.1 Purpose

Provide the entire system with clean, complete, validated, fast local market data. Every downstream module trusts the Data Lake blindly — therefore the Data Lake must earn that trust with aggressive validation.

9.2 Responsibilities

- Bulk-download historical klines (candles) from Binance public archive for the configured universe
- Incrementally update data nightly via CCXT
- Resample base granularity into all higher timeframes deterministically
- Validate data quality (gaps, duplicates, outliers, zero-volume bars, clock integrity)
- Maintain auxiliary datasets: funding rates, mark price (for perp-aware backtests), and a macro event calendar
- Serve data through one function: `get_bars(pair, timeframe, start, end) -> DataFrame`

9.3 Inputs / Outputs

Direction	Artifact	Format
Input	Binance archive ZIPs (data.binance.vision)	Monthly/daily CSV in ZIP
Input	CCXT REST (fetch_ohlcv)	JSON
Output	data/raw/binance/ <PAIR>/<TF>.parquet	Parquet (snappy)
Output	DuckDB views v_bars_<tf>	SQL view
Output	Data quality report (per update)	Markdown/JSON log

9.4 Schema — canonical bar

open_time	TIMESTAMP UTC (bar open, Binance convention, ms→ts)
open	DOUBLE
high	DOUBLE
low	DOUBLE
close	DOUBLE
volume	DOUBLE (base asset)
quote_vol	DOUBLE (quote asset)
trades	INTEGER (count of trades in bar)
taker_buy	DOUBLE (taker buy base volume)
pair	VARCHAR
timeframe	VARCHAR

All downstream code uses exactly these column names. Resampling rule: right-label nothing; bars are indexed by open time; aggregation uses first/max/min/last/sum for O/H/L/C/V and sum for trades, quote_vol, taker_buy.

9.5 Implementation steps

1. **Bulk downloader (src/datalake/binance_archive.py)**: for each pair & month from listing date to now, request `https://data.binance.vision/data/spot/monthly/klines/<PAIR>/<TF>/<PAIR>-<TF>-<YYYY>-<MM>.zip`; stream, unzip in memory, append to Parquet. Use the **1m** klines as the base layer; resample locally to 5m/15m/1h/4h/1d (never download every timeframe — the archive's higher TFs are redundant bytes). Concurrency: 4–8 threads, checksum verify, resume-safe (skip months already stored).
2. **Incremental updater (src/datalake/updater.py)**: nightly APScheduler job → CCXT fetch_ohlcv from last stored timestamp → append → validate.
3. **Validator (src/datalake/validate.py)**: runs after every write; checks listed in §9.7.
4. **Resampler (src/datalake/resample.py)**: vectorized pandas resample; writes per-TF Parquet; unit-tested against hand-computed examples.
5. **Access layer (src/datalake/api.py)**: the ONLY import the rest of the system uses:

```
def get_bars(pair: str, tf: str, start: str, end: str) ->
pd.DataFrame:
    """UTC-indexed OHLCV+ DataFrame from Parquet via DuckDB
    pushdown."""
```

6. **Event calendar (data/calendar/)**: crypto-relevant events — FOMC/CPI dates (macro risk-off), token unlocks for held alts, Binance listing/delisting notices. Start with a manually maintained CSV; automate later.

9.6 Configuration (config/universe.yaml)

```
pairs: [BTCUSDT, ETHUSDT, BNBUSDT, SOLUSDT, XRPUSDT, ADAUSDT,
        DOGEUSDT, AVAXUSDT, LINKUSDT, MATICUSDT, LTCUSDT, DOTUSDT,
        ATOMUSDT, NEARUSDT, ARBUSDT, OPUSDT, INJUSDT, SUIUSDT,
        APTUSDT, FILUSDT]
base_timeframe: 1m
derived_timeframes: [5m, 15m, 1h, 4h, 1d]
history_from: "2017-08-01"
storage: parquet
compression: snappy
```

9.7 Data quality rules (hard failures)

Check	Rule	Action on fail
Gap check	No missing 1m bars beyond exchange maintenance windows (>60 consecutive	Quarantine pair-month; re-download

Check	Rule	Action on fail
	missing bars = investigate)	
Duplicate check	open_time unique per pair/tf	Drop dupes, log count
OHLC sanity	high $\geq$ max(open,close), low $\leq$ min(open,close), all > 0	Quarantine
Outlier check		log-return
Volume check	Zero-volume runs > 30 bars → flag	Investigate
Continuity after resample	Derived 1h bar count == 1m count / 60 ($\pm$ maintenance)	Fail resample
Freshness	After nightly update, latest bar < 24 h old	Alert via Telegram

9.8 Acceptance & Quality Inspection Checklist — Module 1

- ☐ **A1.1** All 20 pairs $\times$ 6 timeframes queryable via `get_bars()`; sample query of 1 year of 15m bars returns in < 2 seconds. *Evidence: timing log.*
- ☐ **A1.2** Spot-check 5 random bars against Binance web UI — exact OHLC match. *Evidence: comparison table.*
- ☐ **A1.3** Gap report exists per pair; total gap fraction $< 0.5\%$ of expected bars; every gap > 60 bars has a written explanation (maintenance, listing date, outage). *Evidence: data_quality_report.md.*
- ☐ **A1.4** Resample unit test passes: a synthetic 1m series with known values produces hand-verified 5m/1h bars. *Evidence: pytest output.*
- ☐ **A1.5** Nightly updater dry-run completes; second run inserts 0 duplicates (idempotent). *Evidence: job log.*
- ☐ **A1.6** Validator runs on full history with zero hard failures; flagged outliers documented. *Evidence: validation JSON.*
- ☐ **A1.7** DuckDB views respond; `SELECT COUNT(*), MIN(open_time), MAX(open_time) FROM v_bars_1h GROUP BY pair` matches expectations per listing date. *Evidence: query output saved.*
- ☐ **A1.8** Disk budget documented: expected size ≈ 20 – 60 GB for 1m history of 20 pairs; actual measured and recorded. *Evidence: du -sh log.*

10. Module 2 — Feature & Regime Factory

10.1 Purpose

Transform raw bars into the labeled, feature-enriched dataset that the miner, backtester, and edge analytics all consume. Regime labels computed here are the dimensions along which your edge filtering happens.

10.2 Feature groups

Group	Features (examples)	Library
Momentum	RSI(2,7,14,21), Stoch, MACD histogram, ROC, Williams %R	pandas-ta
Trend	EMA(9,21,50,200) distance & slope, ADX(14), +DI/-DI, Supertrend, Donchian position	pandas-ta
Volatility	ATR(14) & ATR percentile (252d), Bollinger %B & bandwidth, historical vol (20d), Parkinson vol	pandas-ta / numpy
Volume	Volume z-score (20d), OBV slope, taker-buy ratio, dollar volume	pandas/numpy
Time	hour_utc, day_of_week, is_weekend, week_of_month, month, quarter	pandas
Session-analog*	asia(00-08), europe(07-16), us(12-21), overlap(12-16) UTC labels	custom
Regime — trend	trend_regime $\in$ {up, down, range} from EMA200 slope + ADX threshold	custom
Regime — vol	vol_regime $\in$ {low, mid, high, extreme} from ATR percentile quartiles	custom
Micro-structure	gap vs prior close, distance to 24h high/low, consecutive up/down bars, range position	custom
Funding/perp	funding rate, funding z-score (when using perp data)	archive
Event	minutes_to_event, is_event_day, event_impact	calendar join

*Crypto has no official sessions, but global liquidity still follows the sun: Asia/Europe/US windows are empirically meaningful and give you the same session-edge analysis as forex traders get.

10.3 Regime definitions (pinned, version-controlled)

```
regimes:
  trend:
```

```

up:      "ema200_slope_20bar > 0 AND adx14 >= 22"
down:    "ema200_slope_20bar < 0 AND adx14 >= 22"
range:   "adx14 < 22"
volatility:
  low:    "atr_pctile_252d < 0.25"
  mid:    "0.25 <= atr_pctile_252d < 0.75"
  high:   "0.75 <= atr_pctile_252d < 0.95"
  extreme: "atr_pctile_252d >= 0.95"

```

10.4 Implementation rules

1. **Point-in-time correctness:** feature at bar t may only use data $\leq t$ (no lookahead). Every custom feature gets a lookahead unit test: shift the series by one bar, confirm feature values shift identically.
2. **Compute once, store, reuse:** features are materialized to `data/features/<pair>/<tf>.features.parquet`, rebuilt nightly and versioned (feature_version column) when definitions change.
3. **No NaN leakage:** warm-up rows (first ~252 bars) are marked, not silently dropped mid-pipeline; strategies decide whether to trade warm-up regions.
4. **Deterministic:** same input — same output; no randomness anywhere in this module.

10.5 Acceptance & Quality Inspection Checklist — Module 2

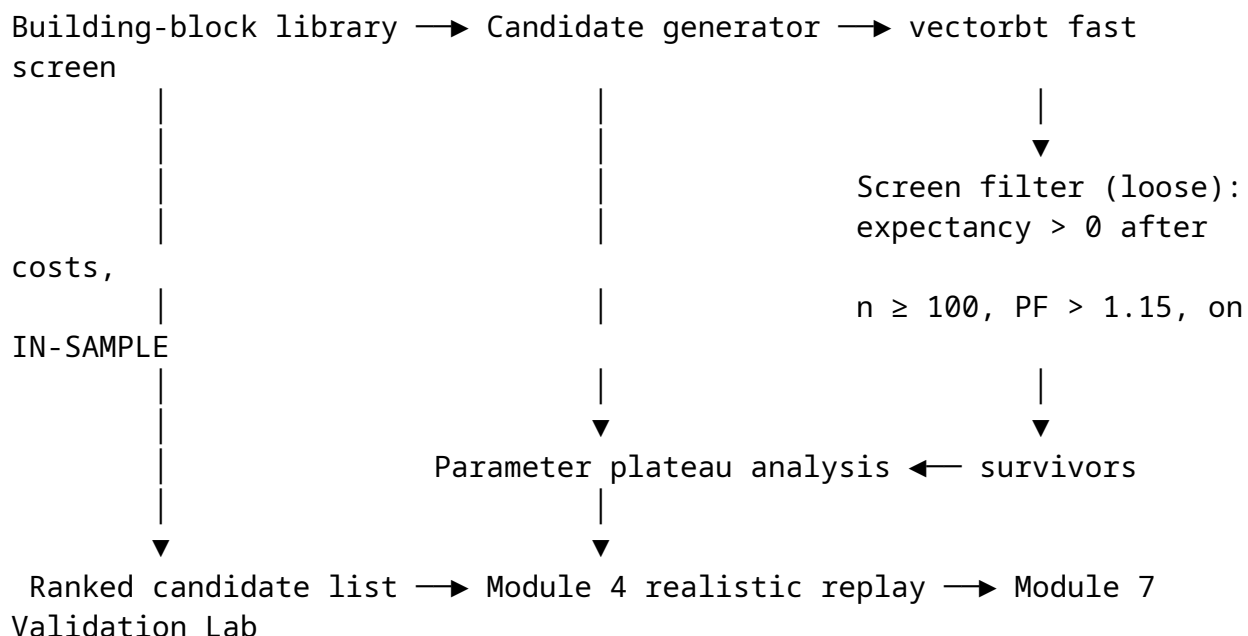
- ☐ **A2.1** Lookahead test suite green for ALL custom features. *Evidence: pytest.*
- ☐ **A2.2** Feature file row count == bar count per pair/tf; join key integrity verified. *Evidence: audit script output.*
- ☐ **A2.3** Regime distribution sanity: across 20 pairs, range $\approx$ 50–70% of bars, low/high vol $\approx$ 25% each by construction of percentiles; extreme < 6%. *Evidence: distribution table.*
- ☐ **A2.4** Spot-verify 3 indicators against TradingView on the same pair/timeframe (RSI, ATR, EMA200) — match within rounding. *Evidence: screenshots + table.*
- ☐ **A2.5** Session labels verified around UTC boundaries (00:00, 07:00, 08:00, 12:00, 16:00, 21:00) with unit tests. *Evidence: pytest.*
- ☐ **A2.6** Changing a regime threshold bumps feature_version; old files retained. *Evidence: version log.*
- ☐ **A2.7** Rebuild time for full universe $\leq$ 30 min on your machine; measured and logged.

11. Module 3 — Strategy Miner

11.1 Purpose

Automate the discovery of candidate strategies. Humans are slow and biased hypothesis generators; the miner explores the combinatorial space systematically and hands the Validation Lab a ranked survivor list.

11.2 Architecture overview



11.3 Miner-1: Brute-force combinatorial miner (build FIRST)

Building blocks (each a versioned function returning a boolean series):

Block type	Examples	Count target (v1)
Entry triggers	EMA cross, RSI threshold cross, BB %B cross, Donchian break, MACD hist flip, volume spike + candle direction, Supertrend flip	20–30
Entry filters	ADX > x, vol_regime ∈ {...}, trend_regime ∈ {...}, session ∈ {...}, day_of_week ∈ {...}, RSI range, funding sign	15–20
Exit models	Fixed SL/TP (R-multiples), ATR trailing, time-stop (N bars), signal-flip exit, partial scale-out	8–12
Direction	long-only, short-only, both	3
Timeframes	5m, 15m, 1h, 4h	4

Search strategy — staged, not full-cartesian (full cartesian of the above is $\sim 10^8$; too much):

1. **Stage 1 — solo scan:** every entry trigger alone × timeframes × pairs ($\sim 30 \times 4 \times 20 = 2,400$ runs). Establish baseline expectancy of each trigger.
2. **Stage 2 — pair-up:** top 30% triggers × every filter × fixed exit ($\sim 10 \times 20 \times 12 \times 4 \times 20 \approx 192,000$ runs — vectorbt handles this overnight).

3. **Stage 3 — exit search:** top 500 combos × all exit models × exit params.
4. **Stage 4 — plateau analysis:** for each survivor, perturb each parameter $\pm 20\%$; score = fraction of neighbors still profitable (plateau score ≥ 0.6 required).

Ranking score (in-sample, pre-validation):

score = expectancy_bps × $\log_{10}(n_trades)$ × plateau_score × consistency
 where consistency = fraction of profitable years in-sample

11.4 Miner-2: Genetic programming miner (Phase 9)

- **Engine:** DEAP (genetic algorithm framework) + gplearn-style expression trees.
- **Genome:** expression tree over feature primitives, e.g. `AND(GT(RSI(14), 68), LT(close, MUL(VWAP, 0.98)))`.
- **Fitness:** OOS-segment-1 Sharpe × $\sqrt{n}$ − $\lambda \times \text{tree_depth}$ — penalize complexity explicitly (λ in config).
- **Anti-overfit mechanics:** (a) fitness evaluated on in-sample sub-segment A, selection pressure from sub-segment B (GeneTrader-style “selection bar”: the distribution of best-of-N luck); (b) max tree depth 6; (c) island model (separate populations, occasional migration) for diversity; (d) final candidates must pass the full Validation Lab like everything else — genetic search is a hypothesis generator, not a shortcut around validation.
- **Compute budget:** overnight runs (~8h); population 500 × 40 generations.

11.5 Miner-3: Local ML layer (Phase 10+, free)

- **Task framing:** predict $P(\text{trade profitable} \mid \text{features at entry})$ using the Trade Database itself as labeled training data — the system literally learns from its own edge mine.
- **Models:** gradient boosting (XGBoost/LightGBM) + logistic regression baseline. SHAP for interpretability (which features drive success — often reveals unknown session/regime structure).
- **Validation:** purged, embargoed time-series CV (López de Prado method) to prevent leakage; predictions only ever trade-sized through the Risk Engine, never naked.
- **Hard rule:** ML output is a *filter/size adjuster* on top of validated strategies, never a standalone black-box strategy, until you have 12+ months of live evidence.

11.6 Mining governance (what makes this professional)

1. Every mining run is registered in runs / with: config hash, git commit, date range, variant count, machine, duration.
2. **Trial budget accounting:** the miner logs `n_variants_tested` — this number feeds the Deflated Sharpe calculation. You cannot compute honest significance without knowing how many things you tried.
3. Survivors are *candidates*, never strategies. Only the Validation Lab can promote a candidate.
4. Miner never sees post-2022 data. Enforced at the data-access layer.

11.7 Acceptance & Quality Inspection Checklist — Module 3

- ☐ **A3.1** Building-block library has ≥ 20 triggers, ≥ 15 filters, ≥ 8 exits, each with unit tests (known input – known signal). *Evidence: pytest + block registry.*
- ☐ **A3.2** Stage-1 solo scan completes for all pairs/timeframes; results table exists with per-trigger baseline stats. *Evidence: DuckDB query.*
- ☐ **A3.3** A 100k-variant Stage-2 run completes overnight; every variant's summary metrics AND every trade logged. *Evidence: run registry + trades count.*
- ☐ **A3.4** Lookahead audit: pick 3 random mined strategies, verify signals computed only from past data. *Evidence: audit notebook.*
- ☐ **A3.5** Plateau analysis implemented; example report shows plateau score distribution; rejection of peak-only configs demonstrated. *Evidence: plateau report.*
- ☐ **A3.6** Trial accounting: n_variants_tested recorded per run and exported for DSR. *Evidence: runs table.*
- ☐ **A3.7** Reproducibility: re-running a registered run with same seed reproduces identical survivor list. *Evidence: diff = empty.*
- ☐ **A3.8** Data-wall test: miner process physically cannot load post-2022 bars (attempt raises exception). *Evidence: failing-access test log.*

12. Module 4 — Backtest & Simulation Engine

12.1 Purpose

Turn candidate strategy configs into realistic, cost-aware simulated trade histories — and log every single trade. Two-speed design: vectorbt for mass screening, an event-driven engine (NautilusTrader) for finalist validation.

12.2 Two-engine contract

Property	vectorbt (screening)	NautilusTrader (finalists)
Speed	~10k variants/hour	~1–10 runs/minute
Cost model	Fees + slippage bps (vectorized)	Per-fill fees, spread, slippage models
Order types	Market	Market/limit/stop/trailing, partial fills
Use	Miner Stages 1–3	Validation Lab, pre-live sign-off
Parity requirement	—	Same strategy must reproduce vectorbt metrics within tolerance ($\pm 10\%$ expectancy) or explain why

12.3 Cost & fill model (never optional)

```
# config/backtest.yaml
```

```
costs:
```

```
  fee_model: taker_bps
```

```
  # v1: all fills taker
```

```

taker_bps: 5                                # per side
slippage_model: fixed_bps
slippage_bps: 2                             # stress-tested at 2x and 3x in MC
fills:
  entry: next_bar_open                      # signal at bar t executes at open of
t+1 – NO same-bar fills
  stop_loss: intrabar_conservative          # if SL and TP both inside bar →
assume SL hit first
  partial_exits: allowed

```

Intrabar ambiguity rule (critical for honesty): with OHLC-only data, if a bar’s range contains both stop and target, the pessimistic outcome (stop first) is recorded. Optimistic intrabar assumptions are the #1 silent backtest liar.

12.4 Run registry (reproducibility spine)

Every backtest execution writes a `runs/<run_id>/` folder:

```

runs/2026-03-14_london-break_v3_eur-style-a7f3/
├─ config.yaml      ← exact strategy + params + data range + costs
├─ metrics.json     ← all summary metrics
├─ equity.parquet   ← bar-by-bar equity
├─ trades_ref.txt   ← DuckDB run_id pointer
├─ git_commit.txt
└─ logs.txt

```

12.5 Metrics computed per run (the full panel)

Total/net return, CAGR, Sharpe, Sortino, Calmar, max drawdown & duration, profit factor, expectancy (bps and R), win rate, avg win/avg loss, payoff ratio, trade count, avg holding time, exposure %, tail ratio, skew, 95th percentile daily loss, monthly returns table, rolling 90d Sharpe min. (quantstats generates the tear sheet; our registry stores the JSON.)

12.6 Acceptance & Quality Inspection Checklist — Module 4

- ☐ **A4.1** Zero-cost parity test: run a trivial known strategy (e.g., buy-and-hold 10 fixed windows) with fees=0; engine matches hand-computed results exactly. *Evidence: test notebook.*
- ☐ **A4.2** Cost impact test: same strategy with 5bps fees shows expectancy drop ≈ 10 bps round-trip ± 1 bp. *Evidence: metrics diff.*
- ☐ **A4.3** No-same-bar-fill test: instrument a strategy whose signal bar also moves favorably; verify fill occurs at $t+1$ open. *Evidence: trade log inspection.*
- ☐ **A4.4** Intrabar-conservative test: synthetic bar hitting both SL and TP → SL recorded. *Evidence: pytest.*
- ☐ **A4.5** vectorbt ↔ Nautilus parity on 3 finalist strategies within tolerance or documented cause. *Evidence: parity report.*

- ☐ **A4.6** Every test run wrote a complete registry folder and its trades appear in DuckDB with matching run_id. *Evidence: query.*
- ☐ **A4.7** Reproducibility: same config re-run → identical trade list hash. *Evidence: hash comparison.*

13. Module 5 — Trade Database (The Edge Mine)

13.1 Purpose

The system's permanent memory and most valuable asset: one analytical store of every trade ever simulated or executed, labeled richly enough that *any* edge question becomes a SQL query.

13.2 Master schema (DuckDB)

```
CREATE TABLE IF NOT EXISTS runs (
  run_id          VARCHAR PRIMARY KEY,    -- date_strategy_hash
  created_at      TIMESTAMP,
  kind            VARCHAR,                --
  'miner1', 'miner2', 'validation', 'wf', 'live', 'manual'
  strategy        VARCHAR,
  params_json     VARCHAR,
  pair            VARCHAR,
  timeframe        VARCHAR,
  data_start      DATE, data_end DATE,
  cost_config     VARCHAR,
  git_commit      VARCHAR,
  seed            INTEGER,
  n_variants      INTEGER,                -- for DSR accounting (miner
runs)
  metrics_json    VARCHAR,
  status          VARCHAR                 --
  'screened', 'validated', 'killed', 'live', 'benched'
);

CREATE TABLE IF NOT EXISTS trades (
  trade_id        BIGINT PRIMARY KEY,
  run_id          VARCHAR REFERENCES runs(run_id),
  strategy        VARCHAR,
  pair            VARCHAR,
  timeframe        VARCHAR,
  direction        VARCHAR,                -- 'long'/'short'
  entry_time       TIMESTAMP, exit_time TIMESTAMP,
  entry_price      DOUBLE,   exit_price DOUBLE,
  qty              DOUBLE,
  pnl_quote        DOUBLE,                -- P&L in USDT
```

```

    pnl_pct      DOUBLE,          -- % on notional
    pnl_r        DOUBLE,          -- P&L in R-multiples
    fees         DOUBLE, slippage DOUBLE,
    mae_pct      DOUBLE, mfe_pct   DOUBLE,    -- excursions
    bars_held    INTEGER,
    exit_reason  VARCHAR,          --
    'tp','sl','trail','time','signal','manual'
    source       VARCHAR,          -- 'backtest','paper','live'
    -- EDGE LABELS (the gold):
    hour_utc     INTEGER, day_of_week INTEGER, week_of_month INTEGER,
    month        INTEGER, session VARCHAR,
    trend_regime VARCHAR, vol_regime VARCHAR,
    rsi_at_entry DOUBLE, adx_at_entry DOUBLE, atr_pctile DOUBLE,
    dist_vwap_pct DOUBLE, funding_z DOUBLE,
    is_event_day BOOLEAN, minutes_to_event INTEGER,
    feature_version VARCHAR
);

CREATE TABLE IF NOT EXISTS edge_cards (
    card_id      BIGINT PRIMARY KEY,
    strategy     VARCHAR, pair VARCHAR,
    filter_json  VARCHAR,          -- the slice definition
    n_trades     INTEGER, expectancy_r DOUBLE, win_rate DOUBLE,
    profit_factor DOUBLE, sharpe DOUBLE,
    in_sample_ok BOOLEAN, oos_ok BOOLEAN, p_value DOUBLE,
    status       VARCHAR,          --
    'active','provisional','retired'
    created_at   TIMESTAMP, last_validated TIMESTAMP
);

CREATE TABLE IF NOT EXISTS live_journal (
    journal_id   BIGINT PRIMARY KEY,
    trade_id     BIGINT,          -- FK to trades
    (source='live')
    planned      BOOLEAN,          -- was it on the edge card?
    checklist_ok BOOLEAN,
    emotion_score INTEGER,          -- 1-5 self rating at entry
    screenshot   VARCHAR,
    notes        VARCHAR,
    created_at   TIMESTAMP
);

```

13.3 Write path

backtest engine → TradeWriter(df) → derive labels (join features at entry_time) → INSERT. Label derivation is centralized in one function (src/tradesdb/label.py) so backtest and live trades get identical labels. Idempotent by (run_id, entry_time, pair) upsert.

13.4 Read path (views)

```
CREATE VIEW v_strategy_summary AS
SELECT strategy, pair, source, COUNT(*) n,
       AVG(pnl_r) exp_r, SUM(pnl_quote) pnl,
       AVG(pnl_r>0)*100 winrate,
       SUM(CASE WHEN pnl_r>0 THEN pnl_r END)/NULLIF(-SUM(CASE WHEN
pnl_r<0 THEN pnl_r END),0) pf
FROM trades GROUP BY 1,2,3;
```

Plus v_hourly, v_daily, v_session, v_regime groupings — prebuilt so the UI and notebooks never write raw GROUP BYs.

13.5 Acceptance & Quality Inspection Checklist — Module 5

- ☐ **A5.1** Schema created idempotently; migrations documented (alembic-style SQL files in db/migrations/). *Evidence: migration log.*
- ☐ **A5.2** Round-trip test: known 50-trade backtest → all 50 rows present, every label non-null (except legitimately null funding for early spot data). *Evidence: audit query.*
- ☐ **A5.3** Label correctness spot-check: 10 random trades' session/day/regime labels verified manually against chart. *Evidence: checklist table.*
- ☐ **A5.4** Idempotency test: re-writing same run adds 0 rows. *Evidence: count before/after.*
- ☐ **A5.5** Backtest/live label parity: a paper trade gets identical label derivation as a backtest trade at same timestamp. *Evidence: comparison test.*
- ☐ **A5.6** Performance: slice query over 500k trades returns < 3s (DuckDB aggregates Parquet fast; if not, partition). *Evidence: timing log.*
- ☐ **A5.7** Backup routine: weekly copy of apex.duckdb + Parquet to second disk/cloud-free-tier; restore tested once. *Evidence: restore drill log.*

14. Module 6 — Edge Analytics Engine

14.1 Purpose

Answer, with statistical honesty: “When, where, and under what conditions does each strategy actually make money?” — and package the answer as actionable Edge Cards.

14.2 Core capabilities

1. **Slice statistics:** arbitrary filter → n, expectancy (R & bps), win rate, profit factor, Sharpe-like ratio, avg win/loss, max consecutive losses, equity curve of slice.

2. **Dimension scans:** auto-loop over (hour × day), (session × regime), (month), (event proximity) producing heatmap tables.
3. **Significance layer (the professional part):**
 - t-test / bootstrap CI on mean R per slice
 - **Multiple-testing correction:** Bonferroni (conservative) or Benjamini–Hochberg FDR across all slices tested in a scan
 - **Stability requirement:** slice must be profitable in BOTH halves of in-sample (2017–2019, 2020–2022)
 - Minimum n: 100 (50–99 = “provisional”, never traded live)

4. **Edge Card generator:** converts a validated slice into the operational card:

EDGE CARD #41 – momo_breakout / SOLUSDT / 15m

TRADE ONLY: session ∈ {europe, us-overlap} AND day ∈ {Tue, Wed, Thu}
AND trend_regime = up AND vol_regime ∈ {mid, high}

SKIP: event days, vol_regime = extreme

STATS: n=412 | exp=+0.31R | WR=47% | PF=1.52 | p<0.01 (BH-adj) |
stable both halves

OOS (2023-25): n=96 | exp=+0.24R ✓ confirmed

14.3 Anti-self-deception protocol (built into the code, not willpower)

- Every scan reports **how many slices were tested** and the adjusted threshold alongside.
- Every Edge Card carries its p-value, n, and both-half stability — cards missing any field cannot be marked active.
- Slices discovered by the miner are re-tested on OOS before activation; post-hoc slices (you noticed a pattern) require a **fresh holdout** or next-3-months forward test before activation.

14.4 Implementation

src/edge/: - slice.py — filter — stats dict - scan.py — dimension iterators — results

DataFrame — Parquet cache - significance.py — tests, corrections, stability splits -

cards.py — Edge Card CRUD + expiry (cards auto-expire 90 days after last validation) - UI exposure in Command Center page “Edge Explorer”

14.5 Acceptance & Quality Inspection Checklist — Module 6

- ☐ **A6.1** Slice engine returns correct stats for a hand-computed 20-trade fixture. *Evidence: pytest.*
- ☐ **A6.2** Hour×Day heatmap for one validated strategy matches independent pandas pivot. *Evidence: diff = 0.*
- ☐ **A6.3** Significance layer: synthetic random-noise trades yield ~0 cards passing at adjusted threshold (false-positive control). *Evidence: simulation report — THIS IS THE KEY TEST.*
- ☐ **A6.4** Synthetic known-edge test: inject artificial edge (e.g., +0.2R on Wednesdays) into noise → engine finds exactly that slice. *Evidence: detection report.*

- ☐ **A6.5** Both-halves stability enforced: a slice profitable only in one half is auto-flagged provisional. *Evidence: unit test.*
- ☐ **A6.6** Edge Card generation end-to-end for 3 real strategies; cards stored with all mandatory fields. *Evidence: DB rows.*
- ☐ **A6.7** Card expiry job works: set last_validated 91 days back → status flips to retired. *Evidence: job log.*

15. Module 7 — Validation Lab (Anti-Overfitting Suite)

15.1 Purpose

The immune system of the whole operation. Its only job: **reject**. A strategy that passes everything here has earned the right to be paper traded; nothing reaches your money without this stamp.

15.2 The gauntlet (in order — cheap tests first, so weak strategies die fast)

Candidate

```

|
├─ GATE 1: OOS replay (2023→now, unseen)    expectancy > 0, PF >
1.1
├─ GATE 2: Walk-forward analysis              WFE ≥ 0.5, no
catastrophic fold
├─ GATE 3: Monte Carlo ×4                    DD CI acceptable,
skip-robust, cost-robust
├─ GATE 4: Regime stress                      survives all 6 regime
cells
├─ GATE 5: PBO (CSCV)                        PBO < 0.20
├─ GATE 6: Deflated Sharpe                   DSR p-value < 0.05
given n_variants tried
└─ VERDICT: VALIDATED / KILLED (with written reason)

```

15.3 Gate specifications

Gate 2 — Anchored walk-forward: - Optimize on [2017–2020] → test [2021]; roll: optimize [2018–2021] → test [2022]... through OOS years; 6+ folds. - Metric: **Walk-Forward Efficiency** = annualized OOS return / annualized IS return. WFE ≥ 0.5 required; also no fold with DD > 1.5× IS max DD. - Output: stitched OOS equity curve — the *only* honest equity curve the strategy has.

Gate 3 — Monte Carlo battery (all four, 10,000 sims each):

MC type	Method	Pass criterion
Trade reshuffle	Random permutation of trade P&L sequence	95th-pct max DD ≤ risk budget (default 20%)
Trade skipping	Drop 10–30% of trades randomly	Median expectancy still > 0
Block bootstrap	Resample contiguous 20-	95th-pct max DD ≤ 25%

MC type	Method	Pass criterion
	trade blocks (preserves streaks)	
Cost stress	Re-run with 2× and 3× fees+slippage	Expectancy > 0 at 2× (3× = informational)

Gate 4 — Regime stress: strategy P&L computed per cell of {trend-up, trend-down, range} × {low-vol, high-vol}; requirement: no cell with expectancy < -0.3R AND $n \geq 30$ (losing mildly in one regime is fine — filters may fix it; catastrophic single-regime dependence is not).

Gate 5 — PBO (Combinatorially Symmetric Cross-Validation, López de Prado): - Split in-sample equity matrix (all parameter variants × time) into 16 blocks; form all $C(16,8)$ train/test combos (12,870). - For each combo: rank variants on train, check where the chosen-best ranks on test (logit of rank). - PBO = fraction of combos where the train-best underperforms test median. **PBO < 0.20 to pass.** This directly answers: “what’s the probability my selected config is overfit?”

Gate 6 — Deflated Sharpe Ratio (Bailey & López de Prado): - Adjusts observed Sharpe for: non-normality (skew/kurtosis), series length, and **number of variants tried** (from the miner’s trial accounting, Module 3). - Pass: DSR > 0 with $p < 0.05$.

15.4 Kill list & archive

Every verdict recorded in `runs.status` with reason code (OOS_FAIL, WF_FAIL, MC_DD, PBO_HIGH, DSR_FAIL, REGIME_FRAGILE). Killed strategies are never deleted — the database of *why things fail* is itself research gold (you’ll notice patterns: “breakouts always die in low-vol regimes”).

15.5 Acceptance & Quality Inspection Checklist — Module 7

- ☐ **A7.1** Walk-forward engine reproduces a hand-rolled 3-fold example exactly. *Evidence: pytest.*
- ☐ **A7.2** MC battery on a known strategy: reshuffle median max-DD within 15% of theoretical bootstrap. *Evidence: comparison report.*
- ☐ **A7.3** Negative control: 100 pure-noise strategies fed to the gauntlet → $\geq 95\%$ killed. *Evidence: control report — if noise passes, the Lab is broken; stop everything.*
- ☐ **A7.4** Positive control: synthetic strategy with injected persistent edge — passes gauntlet. *Evidence: control report.*
- ☐ **A7.5** PBO implementation matches reference example from literature (known test case). *Evidence: unit test.*
- ☐ **A7.6** DSR consumes real `n_variants` from miner registry; verify a 10-variant and 100k-variant trial of same strategy produce different DSR verdicts. *Evidence: demo.*
- ☐ **A7.7** Verdict persistence: every candidate has status + reason; no candidate reaches validated without all 6 gate records. *Evidence: DB audit query.*

16. Module 8 — Portfolio Construction Engine

16.1 Purpose

Decide **how much capital each validated strategy/pair gets**, such that the portfolio survives any single strategy failing. Edge lives in the strategies; survival lives here.

16.2 Methods (via skfolio, compared quarterly)

Method	Use	Notes
Equal weight	Baseline	Always compute; surprisingly hard to beat
HRP (Hierarchical Risk Parity)	Primary	No matrix inversion; robust to estimation error; clusters correlated strategies
Risk parity (vol)	Secondary	Equal risk contribution per strategy
CVaR risk parity	Stress-case	Allocates by tail risk; check in high-vol regimes
Mean-variance (Ledoit-Wolf)	Reference only	Unstable at small n; never deploy raw

Inputs: each strategy's **daily portfolio-return series** (stitched OOS, then live). Estimation window: rolling 252d, minimum 126d. Rebalance: monthly.

16.3 Constraints layer

```
# config/portfolio.yaml
max_weight_per_strategy: 0.30
max_weight_per_pair: 0.40           # strategies on same pair share
risk                                # HRP cluster cap (correlated
max_cluster_weight: 0.50           family)
correlation_guard:
  threshold: 0.60                   # rolling 90d corr of strategy
returns
  action: halve_combined
min_capital_floor: 0.05             # avoid dust allocations
turnover_penalty: true              # prefer stable weights month-to-
month
```

16.4 Capital tiers (because capital starts small)

Tier	Capital	Allocation approach
T1	< \$2,000	Max 2–3 strategies, equal risk weight; HRP meaningless at this size

Tier	Capital	Allocation approach
T2	\$2k–\$25k	HRP over all validated strategies
T3	> \$25k	HRP + CVaR overlay, cluster caps binding

16.5 Acceptance & Quality Inspection Checklist — Module 8

- ☐ **A8.1** HRP weights on 5 synthetic return series match skfolio reference output. *Evidence: test.*
- ☐ **A8.2** Constraints test: feed 10 strategies incl. a correlated pair > 0.6 → combined weight halved, caps respected. *Evidence: allocation report.*
- ☐ **A8.3** Backtest the allocator: historical rebalanced portfolio vs equal-weight — report Sharpe/DD comparison (HRP should show lower DD; if not, investigate). *Evidence: notebook.*
- ☐ **A8.4** Weights always sum to $1.0 \pm 1e-9$; no negative weights. *Evidence: invariant test in CI.*
- ☐ **A8.5** Monthly rebalance job produces dated weight file + diff vs previous month, archived in runs/. *Evidence: files.*

17. Module 9 — Risk Engine

17.1 Purpose

Convert “we have an edge” into “we survive long enough to compound it.” The Risk Engine owns: position size, exposure limits, VaR estimation, and circuit breakers. **Nothing sizes a trade except this module.**

17.2 Position sizing (per trade)

```

risk_per_trade_pct = base_risk × confidence_mult × decay_mult
base_risk          = 0.75% of current equity (range 0.5–1.0% by tier)
confidence_mult    = 0.75 (provisional) | 1.0 (validated) | 1.25 (validated + 100 live trades confirming)
decay_mult         = edge-decay detector output, 0.0–1.0 ($20)

```

```

qty = (equity × risk_per_trade_pct) / (stop_distance_pct)
    then × portfolio_weight (from Module 8)

```

Optional upgrade (after 6 months of data): **volatility targeting** — scale all sizes so forecast portfolio vol (GARCH via arch) $\approx 12\%$ annualized; fractional-Kelly ($\frac{1}{4}$) cross-check cap.

17.3 Exposure & circuit breakers (hard limits, code-enforced)

Breaker	Limit	Action
Per-trade risk	$\leq 1\%$ equity	Reject order plan
Concurrent positions	≤ 5	Queue new signals

Breaker	Limit	Action
Same-direction single-asset exposure	≤ 2 positions	Skip correlated adds
Daily loss	-1.5% equity	Flat + locked until next day
Weekly loss	-4%	Flat + locked until Monday
Strategy drawdown	-15% from its peak	Strategy benched → re-validation
Portfolio drawdown	-12%	All sizes halved
Portfolio drawdown	-20%	FULL STOP → manual review, re-validate everything
API/journal failure	—	No new trades until logging restored

17.4 Risk metrics (dashboard + weekly report)

- Historical VaR/CVaR at 95% & 99% (daily)
- GARCH(1,1) forecast vol & parametric VaR (arch)
- Rolling 30d/90d Sharpe, Sortino, Calmar
- Current drawdown vs Monte Carlo DD distribution (“current DD is at the 78th percentile of expected” — context, not panic)
- Risk contribution per strategy (should roughly match HRP intent)

17.5 Acceptance & Quality Inspection Checklist — Module 9

- ☐ **A9.1** Sizing math verified by hand on 10 examples (equity $\times$ risk / stop distance $\times$ weight). *Evidence: test sheet.*
- ☐ **A9.2** Breaker tests: simulate -1.5% day → lockout triggers and logs; simulate strategy DD -15% → bench event written. *Evidence: simulation log.*
- ☐ **A9.3** VaR backtest: on 2 years of portfolio returns, exceptions of 95% VaR $\approx 5\% \pm 1.5\%$ of days (Kupiec test). *Evidence: backtest report.*
- ☐ **A9.4** GARCH forecast vs realized vol scatter is sane (correlation > 0.5 on 20 pairs). *Evidence: notebook.*
- ☐ **A9.5** Decay multiplier wiring: force decay_mult=0 → zero size; force 0.5 → half size. *Evidence: integration test.*
- ☐ **A9.6** Every breaker event auto-posts to Telegram with reason and unlock condition. *Evidence: test message.*

18. Module 10 — Command Center (UI)

18.1 Purpose

One screen where you *operate* the system. The UI’s job is decision compression: convert 100k trades and a dozen models into “here is exactly what you may trade today, at what size, and what would invalidate it.”

18.2 Stack decision

Streamlit (primary). Rationale: pure Python (talks to DuckDB natively), fast to build, runs locally, free, reactive. A React+FastAPI rebuild is a Phase-G luxury (\$31) — only after the system is profitable enough to justify it.

18.3 Page specifications

Page 1 — Overview (the daily screen) - Portfolio equity curve (live vs backtest expectation band) - Today's active Edge Cards (strategy, pair, window, conditions, computed size) - Risk status: current DD, daily loss used, breakers armed/tripped, VaR gauge - System health: last data update, last mining run, alerts pending

Page 2 — Miner Control - Launch mining runs (stage, pairs, timeframes, budget) with live progress - Survivor leaderboard with plateau scores - Run history: variants tested, survival funnel (tested → screened → validated → live)

Page 3 — Edge Explorer - Filter panel: strategy / pair / hour / day / session / regime / exit reason / source (backtest|paper|live) - Outputs: slice stats, equity curve, heatmaps (hour×day), distribution of R - Significance badges (n, p-adj, stability) on every slice — no naked numbers

Page 4 — Validation Center - Per strategy: walk-forward folds chart, MC cloud (10k equity paths + DD histogram), PBO score, DSR, regime grid - Gate-by-gate pass/fail board; kill-list browser with reasons

Page 5 — Portfolio - Current weights (HRP), correlation heatmap, cluster tree (dendrogram), risk contributions, rebalance diff preview

Page 6 — Trade Journal - Filterable live trade table; per-trade detail incl. checklist state, emotion score, screenshot - Live-vs-backtest expectancy drift chart per strategy

Page 7 — Data & System Manager - Coverage matrix (pair × timeframe × date), gap report, last-update timestamps - Job scheduler status; disk usage; config viewer (read-only YAML)

18.4 Design standards

- One accent color family (deep teal/slate), dark theme default, zero clutter
- Every number that drives a trading decision carries its n and timeframe in-line
- All charts Plotly (zoomable); every table downloadable as CSV
- Pages must load < 5s on 500k-trade DB (pre-aggregate via DuckDB views, cache with `@st.cache_data`)

18.5 Acceptance & Quality Inspection Checklist — Module 10

- ☐ **A10.1** All 7 pages load against a 500k-trade fixture DB in < 5s each. *Evidence: timing log.*
- ☐ **A10.2** Overview numbers cross-verified against raw SQL (equity, DD, card list) — exact match. *Evidence: comparison.*

- ☐ **A10.3** Edge Explorer slice stats match Module-6 engine output for 5 random filters. *Evidence: diff test.*
- ☐ **A10.4** No decision number displayed without n and period. *Evidence: manual page audit.*
- ☐ **A10.5** UI survives empty states gracefully (new DB, no live trades yet) — no crashes/stack traces. *Evidence: test with empty DB.*
- ☐ **A10.6** Read-only guarantee: UI contains no write paths to trading tables except the journal form. *Evidence: code review.*

19. Module 11 — Execution & Live Loop

19.1 Purpose

Bridge from analysis to action. Three maturity stages, each gated:

Stage	Mode	Gate to advance
S1	Manual (system alerts — you click on exchange)	8 weeks + 50 paper trades matching backtest within 1.5σ
S2	Semi-auto on Binance testnet (Freqtrade or CCXT bot)	8 weeks testnet, fills/slippage within model
S3	Live auto-execution (small capital)	S2 green + full breaker stack + kill-switch drill passed

19.2 Stage-1 manual protocol (the operating loop)

1. **Pre-session (5 min):** open Overview; read today's Edge Cards; note invalidation conditions; Risk Engine shows your size per card.
2. **Alert:** APScheduler + Telegram bot fires at window open: *"EDGE #41 active now — SOLUSDT 15m momo_breakout, conditions met (ADX 27, vol mid). Size: 0.75% risk = 1.24 units. Invalid below 142.30."*
3. **Pre-trade checklist (in the journal form, mandatory):** conditions verified? news blackout clear? daily loss limit untouched? size from Risk Engine (not gut)? emotion score logged?
4. **Execute** at market/limit; set stop immediately.
5. **Post-trade:** journal the fill (the journal writes to trades with source='live'); optional chart screenshot.
6. **No-card-no-trade law:** a trade without an active Edge Card is a protocol violation — journal marks it; three violations in a week — system locks you out for the week (yes, build this).

19.3 Slippage & fill calibration

Monthly: compare live fill prices vs alert-time mid → measured slippage distribution → update config/backtest.yaml slippage assumptions. The backtest cost model must track reality, not hopes.

19.4 Acceptance & Quality Inspection Checklist — Module 11

- ☐ **A11.1** Alert bot delivers test messages with correct card content and sizing. *Evidence: screenshot + DB row.*
- ☐ **A11.2** Journal form writes complete trade + journal rows; rejects incomplete checklist. *Evidence: test submission.*
- ☐ **A11.3** Violation counter: 3 un-carded trades — lockout state set, Overview shows lock banner. *Evidence: integration test.*
- ☐ **A11.4** Testnet bridge (S2): bot places/closes 10 trades on Binance testnet; all journaled; measured vs expected slippage reported. *Evidence: report.*
- ☐ **A11.5** Kill-switch drill: one command/message flattens everything and disarms alerts; drill log signed. *Evidence: drill log.*
- ☐ **A11.6** Monthly slippage calibration report generated from journal data. *Evidence: report file.*

20. Module 12 — Monitoring, Alerts & Edge-Decay Detection

20.1 Purpose

Markets change; edges die quietly. This module watches every strategy's live performance against its own history and benches decaying edges before they do damage.

20.2 Edge-decay detector

For each live strategy, maintain: - **Rolling expectancy**: trailing 30-trade mean R vs backtest distribution → z-score. $z < -1.5$ → warning; $z < -2$ → decay_mult halves size; $z < -2.5$ sustained 20 trades → auto-bench → re-validation required. - **Regime context**: is the strategy's favored regime simply absent (e.g., a trend strategy in a 2-month range)? Distinguish *regime absence* (wait) from *edge death* (bench). - **Time-since-validation**: any Edge Card > 90 days unvalidated auto-expires (Module 6).

20.3 Automated reports

Report	Cadence	Contents
Daily digest	EOD Telegram	P&L, trades taken vs cards, breaker status, tomorrow's windows
Weekly report	Sunday	Per-strategy live-vs-backtest drift, decay z-scores, risk metrics, journal adherence %
Monthly deep-dive	1st	Full quantstats tear sheets, rebalance proposal, re-validation queue, miner summary, data health

20.4 System health monitoring

- Data freshness (last bar age), job success/failure log, DB size growth, disk space
- Any failed nightly job → Telegram alert before you'd notice stale cards

20.5 Acceptance & Quality Inspection Checklist — Module 12

- ☐ **A12.1** Decay detector catches synthetic decay: feed a strategy's live stream degraded by $-0.3R/\text{trade}$ — warning within 30 trades, bench within 50. *Evidence: simulation.*
- ☐ **A12.2** Regime-absence vs edge-death classification demonstrated on 2 historical cases. *Evidence: notebook.*
- ☐ **A12.3** All three reports generate correctly with real data; delivered via Telegram/files on schedule. *Evidence: received reports.*
- ☐ **A12.4** Health monitor flags stale data (simulate by freezing updater) within one cycle. *Evidence: test alert.*
- ☐ **A12.5** Every bench/warning event writes to DB with reason code and timestamp. *Evidence: audit query.*

PART C-2 — BUILDER'S COOKBOOK

This part contains the detailed reference material you will keep open while coding: every data column defined, every strategy building block specified, eight complete starter strategies, a full worked example, the contracts between modules, the testing patterns, and the operational templates.

C2.1 Data Dictionary & Feature Catalog

C2.1.1 Bar table columns (canonical)

Column	Type	Unit	Definition	Notes
open_time	TIMESTAMP	UTC ms→ts	Bar open timestamp	Binance convention: bar covers [open_time, open_time+tf)
open	DOUBLE	USDT	First trade price in bar	—
high	DOUBLE	USDT	Highest trade price in bar	Must be $\geq \max(\text{open}, \text{close})$
low	DOUBLE	USDT	Lowest trade price in bar	Must be $\leq \min(\text{open}, \text{close})$
close	DOUBLE	USDT	Last trade price in bar	—
volume	DOUBLE	base asset	Sum of base-asset volume	e.g., BTC for BTCUSDT
quote_vol	DOUBLE	USDT	Sum of quote-asset volume	Liquidity proxy
trades	INTEGER	count	Number of trades in bar	Activity proxy

Column	Type	Unit	Definition	Notes
taker_buy	DOUBLE	base asset	Taker-buy base volume	buy/sell pressure: ratio = taker_buy/volume

C2.1.2 Time & session features

Feature	Values	Definition
hour_utc	0–23	Hour of bar open, UTC
day_of_week	0–6	Monday=0 ... Sunday=6
is_weekend	bool	Sat/Sun — crypto trades weekends, but liquidity/participation differs; test as filter
week_of_month	1–5	Calendar week within month
month	1–12	Calendar month (seasonality scans)
session	asia / europe / us / overlap / off	UTC windows: asia 00–07, europe 07–12, overlap 12–16, us 16–21, off 21–24
minutes_from_session_open	int	Bars-since current session window opened
is_month_end	bool	Last 2 trading days of month (rebalancing flows)
minutes_to_event	int/null	Minutes to next scheduled high-impact event
is_event_day	bool	Any high/medium event scheduled that UTC day

C2.1.3 Momentum features

Feature	Params	Formula / Definition	Typical use
rsi_2 / rsi_7 / rsi_14 / rsi_21	Wilder	RSI over N bars	MR triggers (2,7), trend filter (14,21)
stoch_k / stoch_d	14,3	$\%K = (C - LL14) / (HH14 - LL14) \times 100$	MR within ranges
macd_hist	12,26,9	MACD line – signal line	Momentum flips
roc_10	10	$(C/C[-10] - 1) \times 100$	Breakout strength
willr_14	14	Williams %R	OB/OS in ranges
cci_20	20	Commodity Channel Index	Cyclical extremes

C2.1.4 Trend features

Feature	Definition
ema_9/21/50/200	Exponential moving averages of close
ema200_slope_20	$(\text{ema200} - \text{ema200}[-20]) / \text{ema200}[-20] \times 100$ — normalized trend direction
adx_14, plus_di, minus_di	Wilder ADX family; trend strength & direction
supertrend_dir	± 1 from Supertrend(10, 3)
donchian_pos	$(C - \text{LL20}) / (\text{HH20} - \text{LL20})$, 0–1 channel position
dist_ema200_pct	$(C / \text{ema200} - 1) \times 100$ — extension measure

C2.1.5 Volatility & volume features

Feature	Definition
atr_14	Wilder ATR
atr_pct	$\text{atr_14} / \text{close} \times 100$ — normalized vol
atr_pctile_252d	Percentile of current atr_pct within trailing 252 days (0–1)
bb_pctb	$(C - \text{lowerBB}) / (\text{upperBB} - \text{lowerBB})$, BB(20,2)
bb_width	$(\text{upper} - \text{lower}) / \text{mid}$ — squeeze detection
hv_20	20-bar stdev of log returns $\times \sqrt{\text{periods}}$
parkinson_20	Range-based vol estimator
vol_z_20	$(\text{volume} - \text{mean20}) / \text{std20}$ — volume shock
obv_slope_10	Slope of On-Balance-Volume over 10 bars
dollar_vol_20d	20-day mean quote_vol — liquidity floor filter

C2.1.6 Micro-structure & regime features

Feature	Definition
gap_pct	$(\text{open} - \text{prev_close}) / \text{prev_close} \times 100$ (4h/1d timeframes)
dist_24h_high/low_pct	Distance to trailing 24h extremes
consec_bars	Consecutive same-direction closes (signed)
range_pos	Position of close within current bar range, 0–1
trend_regime	up / down / range — §10.3 rules
vol_regime	low / mid / high / extreme — §10.3 quartiles
funding_rate / funding_z	Perp funding and its 90d z-score (perp data only)

Feature versioning rule: any change to a formula in this catalog bumps feature_version (semver). Trade rows store the version used at their entry time, so old analyses remain interpretable forever.

C2.2 Strategy Building-Block Catalog

Every block is a pure function: (bars_df, features_df, params) -> boolean Series (triggers/filters) or (position, bars, features, params) -> exit signal (exits). All blocks are unit-tested against hand-computed fixtures.

C2.2.1 Entry triggers (v1: 24 blocks)

ID	Name	Logic	Params
T01	EMA cross up/down	ema_fast crosses above ema_slow	fast $\in$ {5..21}, slow $\in$ {21..200}
T02	RSI threshold cross	rsi_n crosses above/below level	n $\in$ {2,7,14}, level $\in$ {10..90}
T03	RSI pullback in trend	rsi_2 < 10 while trend_regime=up (long)	rsi level, trend gate
T04	BB %B cross	%B crosses below 0 / above 1	bb_len, bb_std
T05	Donchian breakout	close > HH_n of prior bars	n $\in$ {10,20,55}
T06	MACD hist flip	hist crosses 0	standard 12/26/9
T07	Volume-shock momentum	vol_z > z AND candle direction	z $\in$ {2,3}
T08	Supertrend flip	direction changes	(10, mult $\in$ {2,3,4})
T09	Session-open range break	break of first-hour range of session	session, buffer_bps
T10	Gap fill	gap > x% then reversal back into gap	gap threshold
T11	Consecutive-bars reversal	consec_bars $\geq$ k then first opposite close	k $\in$ {3..6}
T12	Momentum persistence	roc_10 in top decile $\rightarrow$ continuation	lookback, threshold
T13	VWAP stretch revert	dist_vwap beyond σ band $\rightarrow$ revert	σ mult $\in$ {1.5,2,2.5}
T14	Funding extreme fade	funding_z extreme $\rightarrow$ fade perp crowd	z $\in$ {2,2.5,3}
T15	Range-position snapback	close in bottom 10% of bar after down move	filters required

ID	Name	Logic	Params
T16	24h-high/low retest	touch & rejection wick	wick ratio
T17	BB squeeze break	bb_width in bottom quintile – trade the break	width pctile
T18	Stoch cross in range	%K×%D cross while trend_regime=range	14,3
T19	CCI extreme revert	cci_20 beyond ±100 revert	level
T20	ATR expansion momentum	atr_pct jump > x% with direction	expansion factor
T21	EMA200 first-touch	first touch of ema200 after >50 bars away	distance memory
T22	Multi-TF alignment	HTF trend + LTF trigger	htf gate + any trigger
T23	Wick rejection	lower wick > 2× body at support zone	wick/body ratio
T24	Month-end drift	directional bias into month close	calendar gate

C2.2.2 Filters (v1: 18 blocks)

ID	Name	Passes when
F01	Trend gate	trend_regime ∈ chosen set
F02	Vol gate	vol_regime ∈ chosen set
F03	ADX floor/ceiling	adx_14 > x (or < x)
F04	Session gate	session ∈ chosen set
F05	Day gate	day_of_week ∈ chosen set
F06	Hour gate	hour_utc ∈ chosen ranges
F07	Weekend toggle	is_weekend = chosen
F08	Event blackout	minutes_to_event > buffer AND not is_event_day
F09	Liquidity floor	dollar_vol_20d > threshold
F10	EMA200 side	close on chosen side of ema200
F11	RSI zone	rsi_14 within range
F12	Funding side	funding sign matches direction logic
F13	Squeeze gate	bb_width pctile in chosen band

ID	Name	Passes when
F14	ATR band	atr_pct within min-max (avoid dead & insane vol)
F15	Correlated-position guard	no open same-direction trade on correlated pair
F16	Cooldown	≥ N bars since last trade on same pair
F17	Month-end toggle	is_month_end = chosen
F18	Slope gate	ema200_slope_20 sign matches direction

C2.2.3 Exit models (v1: 10 blocks)

ID	Name	Logic	Params
X01	Fixed SL/TP	stop at $k \times \text{ATR}$, target at $m \times \text{ATR}$	k, m
X02	R-multiple	stop 1R, target $r \times R$	$r \in \{1, 1.5, 2, 3\}$
X03	ATR trailing	chandelier: highest_high - $k \times \text{ATR}$	$k \in \{2, 3, 4\}$
X04	Time stop	exit after N bars regardless	$N \in \{8..96\}$
X05	Signal flip	exit on opposite trigger	—
X06	Scale-out	50% at 1R, stop → BE, trail rest	ratios
X07	Session close	exit at session window end	session
X08	RSI normalize	exit when rsi crosses back through 50	rsi_n
X09	Donchian mid exit	exit at channel midline	n
X10	Vol-crush exit	exit if vol_regime collapses to low while in trade	—

Block governance: adding a block requires (a) unit test, (b) documentation row in this catalog, (c) solo-scan baseline run. Blocks that never appear in any surviving strategy after 6 months get archived — keeps the search space honest.

C2.3 Starter Strategy Playbook

Eight hand-built strategies to seed the system before the miner takes over. Each is specified completely enough to code without further design decisions.

S1 — Session-Open Momentum Breakout (Momentum)

- **Hypothesis:** liquidity and directional flows concentrate after session opens; breaks of the opening range continue.
- **Universe/TF:** BTCUSDT, ETHUSDT, SOLUSDT; 15m.
- **Setup:** define first-hour range of europe session (07:00–08:00 UTC).
- **Entry (long):** close above range high + $0.1 \times \text{ATR}$ buffer, within 3 hours of range close. Short symmetric.
- **Filters:** $\text{ADX} > 20$; $\text{vol_regime} \in \{\text{mid}, \text{high}\}$; event blackout 30 min; no weekend.
- **Exit:** $\text{SL} = 1 \times \text{ATR}$; $\text{TP} = 2R$; time stop 12 bars; hard exit at session end.
- **Expected profile:** $\text{WR} \sim 38\text{--}45\%$, $\text{PF} \sim 1.3\text{--}1.5$, 8–15 trades/pair/month.

S2 — VWAP Stretch Reversion (Mean Reversion)

- **Hypothesis:** price over-extends from session VWAP and mean-reverts intraday.
- **Universe/TF:** top-10 liquid pairs; 5m/15m.
- **Entry (long):** $\text{close} \leq \text{VWAP} - 2\sigma(\text{VWAP bands})$ AND wick rejection ($\text{range_pos} < 0.25$). Short symmetric.
- **Filters:** $\text{trend_regime} = \text{range}$; $\text{vol_regime} \in \{\text{low}, \text{mid}\}$; event blackout; session $\in \{\text{europe}, \text{us}\}$.
- **Exit:** $\text{TP} = \text{VWAP touch}$; $\text{SL} = 1 \times \text{ATR}$ beyond extreme; time stop 16 bars.
- **Profile:** $\text{WR} \sim 55\text{--}65\%$, small avg win, needs tight cost control.

S3 — RSI-2 Pullback in Trend (Mean Reversion, Connors-style)

- **Hypothesis:** sharp short-term dips within uptrends get bought.
- **Universe/TF:** 10 pairs; 1h.
- **Entry (long):** $\text{close} > \text{ema}_{200}$ AND $\text{rsi}_2 < 5 \rightarrow$ buy next open. Short: $\text{close} < \text{ema}_{200}$ AND $\text{rsi}_2 > 95$.
- **Filters:** $\text{vol_regime} \neq \text{extreme}$; F16 cooldown 24h.
- **Exit:** $\text{rsi}_2 > 70$ (long) or time stop 48 bars; $\text{SL} 2 \times \text{ATR}$.
- **Profile:** $\text{WR} \sim 60\%+$, holds hours–2 days.

S4 — Donchian Trend-Following (Trend)

- **Hypothesis:** long lookahead breakouts capture multi-day trends; classic turtle logic works on majors.
- **Universe/TF:** 20 pairs (breadth matters here); 4h/1d.
- **Entry:** close beyond 55-bar Donchian extreme.
- **Filters:** ADX rising; dollar_vol floor; none else (trend systems want few filters).
- **Exit:** 20-bar opposite Donchian touch OR $3 \times \text{ATR}$ chandelier trail.
- **Profile:** $\text{WR} \sim 30\text{--}38\%$, fat right tail; the portfolio's regime-diversifier.

S5 — Funding-Extreme Fade (Sentiment)

- **Hypothesis:** extreme positive funding = overcrowded longs $\rightarrow$ fade; symmetric for negative.
- **Universe/TF:** perp markets of 10 pairs; 1h/4h.

- **Entry (short):** funding_z > 2.5 AND price fails to make new high on the funding print. Long symmetric.
- **Filters:** vol_regime ∈ {mid, high}; trend_regime ≠ strong trend against fade.
- **Exit:** funding normalizes ($|z| < 0.5$) or 2×ATR stop; time stop 72h.
- **Profile:** event-driven, 5–10 trades/pair/month.

S6 — Post-Event Drift (Event)

- **Hypothesis:** after scheduled macro events (FOMC/CPI), the initial move continues for hours.
- **Universe/TF:** BTC, ETH; 5m/15m.
- **Entry:** 30 min after event, direction = sign of post-event return if $|\text{move}| > 1 \times \text{ATR}(1d) / \sqrt{24}$.
- **Filters:** only high-impact events; spread normal; no chop (first 30 min range not reversing).
- **Exit:** TP 1.5R; SL 1R; hard time stop end of us session.
- **Profile:** ~2–4 events/month; small n — needs years of data to validate.

S7 — Range Oscillator (Mean Reversion)

- **Hypothesis:** in confirmed ranges, stochastics extremes revert.
- **Universe/TF:** 10 pairs; 15m/1h.
- **Entry (long):** trend_regime=range AND stoch_k < 15 AND stoch_k crosses above stoch_d. Short symmetric at 85.
- **Filters:** bb_width pctile < 60 (actually ranging); session ≠ asia-thin hours optional (test).
- **Exit:** mid-BB or stoch opposite extreme; SL 0.8×ATR; time stop 24 bars.

S8 — Month-End Flow Drift (Calendar)

- **Hypothesis:** institutional rebalancing creates directional drift in the last 2 days of month.
- **Universe/TF:** BTC, ETH; 1h/4h.
- **Entry:** direction = sign of month's return (continuation), entered on second-to-last day.
- **Filters:** month's $|\text{return}| > 5\%$; no event blackout clash.
- **Exit:** end of first day of new month; SL 1.5×ATR.
- **Profile:** only ~1 trade/pair/month — validate on full history; small n, treat as provisional forever unless cross-market confirmed.

Playbook governance: these eight are *hypotheses with specifications*, not truths. Each enters the same Validation Lab gauntlet as miner output. Expect 3–5 of the 8 to survive — and expect the miner's discoveries to replace most of them within a year.

C2.4 Worked Example — End-to-End Walkthrough

How one idea travels through the entire system. Use this as the reference when building each module — your implementation should reproduce this journey.

Step 1 — Mining (Miner-1, Stage 2). Trigger T09 (session-open range break) paired with 18 filters × 10 exits × 3 pairs × 15m: 5,400 variants screened overnight. 214 pass the loose screen (expectancy > 0 after costs, $n \geq 100$, PF > 1.15).

Step 2 — Plateau analysis. Variant T09+F03 (ADX>20)+F08+X01 ($k=1, m=2$) on ETHUSDT shows: expectancy +0.22R, $n=631$ in-sample, and 14/16 parameter neighbors also profitable → plateau score 0.875. Promoted to candidate.

Step 3 — Realistic replay (Module 4). NautilusTrader run with per-fill fees + slippage: expectancy drops to +0.19R (within tolerance), 629 trades logged to DuckDB with run_id 2026-05-02_T09eth_8c1e.

Step 4 — Edge analysis (Module 6). Hour×day scan finds: expectancy +0.34R during europe+overlap sessions Tue–Thu vs +0.02R otherwise; session filter passes BH correction ($p_{\text{adj}}=0.003$), stable in both halves → Edge Card #EC-104 drafted:

TRADE ONLY: session ∈ {europe, overlap}, day ∈ {Tue, Wed, Thu},
ADX>20, vol_regime ∈ {mid, high}; event blackout 30m.

Step 5 — Validation Lab. - Gate 1 OOS (2023–25): $n=187$, exp +0.21R, PF 1.38 ✓ - Gate 2 walk-forward: WFE = 0.71, worst fold DD 1.1× IS ✓ - Gate 3 MC: 95th-pct DD 13% (budget 20%) ✓; skip-30% median exp +0.13R ✓; 2× costs exp +0.09R ✓ - Gate 4 regime grid: range×low-vol cell -0.08R (acceptable) ✓ - Gate 5 PBO = 0.11 ✓ · Gate 6 DSR $p=0.02$ ($n_{\text{variants}}=5,400$) ✓ -

Verdict: VALIDATED. Status → validated, Edge Card #EC-104 → active.

Step 6 — Portfolio & risk. HRP assigns 14% weight; Risk Engine sizes at $0.75\% \times 1.0 \times 1.0$ → per-trade risk \$7.50 on \$1,000 paper equity.

Step 7 — Paper trading. 11 weeks, 41 paper trades: exp +0.16R vs backtest +0.21R (within 1.5σ) → advances to live micro.

Step 8 — Live & monitoring. Month 3 live: decay $z = -0.4$ (healthy). Monthly re-validation keeps card active. This loop then repeats every month, automatically.

C2.5 Module Interface Contracts

These function signatures are the system’s “API.” Modules may only talk through them.

Data Lake

```
get_bars(pair: str, tf: str, start: str, end: str) -> pd.DataFrame
get_events(start: str, end: str) -> pd.DataFrame
data_quality_report(pair: str|None = None) -> dict
```

Feature Factory

```
get_features(pair: str, tf: str, start: str, end: str,
             version: str = "latest") -> pd.DataFrame # joinable on
open_time
```

Miner

```

run_mining_stage(stage: int, config_path: str) -> str # returns
run_id
plateau_analysis(candidate_ids: list[str]) -> pd.DataFrame

# Backtest Engine
run_screen(strategy_cfg: dict, data: pd.DataFrame) -> dict #
vectorbt summary
run_realistic(strategy_cfg: dict, pair: str, start: str, end: str) ->
str # → run_id

# Trade DB
write_trades(run_id: str, trades_df: pd.DataFrame) -> int #
rows written
query(sql: str, params: dict = None) -> pd.DataFrame

# Edge Analytics
slice_stats(filter_dict: dict) -> dict
scan_dimensions(strategy: str, dims: list[str]) -> pd.DataFrame
make_edge_card(strategy: str, filter_dict: dict) -> int #
card_id

# Validation Lab
run_gauntlet(strategy: str, run_id: str) -> dict #
gates + verdict

# Portfolio Engine
compute_weights(method: str = "hrp", as_of: str = None) -> pd.Series

# Risk Engine
size_position(strategy: str, pair: str, stop_distance_pct: float) ->
dict
check_breakers() -> list[dict] #
active events

# Execution
publish_edge_cards(date: str) -> list[dict]
journal_trade(payload: dict) -> int #
trade_id

# Monitoring
decay_status(strategy: str = None) -> pd.DataFrame
send_alert(channel: str, message: str) -> bool

```

Contract rules: (1) no module imports another's internals — only these functions; (2) every function logs its calls; (3) breaking a contract requires updating this document + all callers in the same commit.

C2.6 Testing Cookbook

C2.6.1 The no-lookahead test pattern (use for every feature/block)

```
def test_no_lookahead(feature_fn, sampleBars):
    full = feature_fn(sampleBars)
    for cut in [100, 500, len(sampleBars) - 10]:
        partial = feature_fn(sampleBars.iloc[:cut])
        pd.testing.assert_series_equal(
            full.iloc[:cut - 1], partial.iloc[:cut - 1],
            check_names=False)    # last partial bar may be incomplete
```

Any failure = the feature peeks into the future = would have poisoned every backtest using it.

C2.6.2 The synthetic-fixture pattern (use for every indicator)

Build tiny series by hand where the answer is known:

```
bars = makeBars(closes=[10, 11, 12, 13, 14])    # steady rise
assert rsi(bars, n=2).iloc[-1] == pytest.approx(100.0, abs=1e-6)
```

Five such fixtures per indicator beat 500 lines of “looks right” plotting.

C2.6.3 The known-strategy parity pattern (engines)

Buy-and-hold over 10 fixed windows with fees=0 must match $qty \times (exit - entry)$ to the cent. With fees=5bps it must be exactly $2 \times \text{notional} \times 0.0005$ lower. Any engine that can't reproduce arithmetic can't be trusted with strategies.

C2.6.4 The negative-control pattern (miner, edge engine, validation lab)

```
rng = np.random.default_rng(42)
noise_trades = pd.DataFrame({"pn1_r": rng.normal(0, 1,
5000), ...labels...})
cards = scan_dimensions(noise_trades, dims=["hour_utc",
"day_of_week"])
assert (cards.p_adj < 0.05).mean() < 0.02    # false-positive rate
under control
```

Run after ANY change to statistics code. If random data produces “edges,” the lab is broken and every prior result is suspect.

C2.6.5 The positive-control pattern

Inject a known edge (e.g., +0.25R whenever `day_of_week == 2`) into noise. The edge engine must surface exactly `day=2` in its top results; the validation lab must pass the synthetic strategy. If the system can't find a *planted* edge, it won't find real ones.

C2.6.6 The idempotency pattern (writers, updaters)

```
n1 = write_trades(run_id, trades)  # first write
n2 = write_trades(run_id, trades)  # same write again
assert n1 == len(trades) and n2 == 0
```

C2.6.7 Test data management

- `tests/fixtures/` holds small frozen Parquet slices (3 pairs × 6 months × 15m) — enough for integration tests, fast enough for CI
- Never test against the full data lake; never let tests write outside `tests/tmp/`
- Fixtures are regenerated only via a scripted, reviewed procedure (they're the ground truth — treat them like it)

C2.6.8 Coverage & CI policy

- `pytest --cov=src` on every push; core modules (datalake, tradesdb, edge, risk, validation) ≥ 80%
- Statistical controls run nightly (they're slow), not per-commit
- A red pipeline blocks merges. No “I'll fix it later” — later never comes

C2.7 Operational Playbooks & Incident Response

Pre-written responses so bad days have procedures, not panic.

PB-1: Daily-loss breaker fired 1. Breaker flattens and locks (automatic). 2. Do not override. 3. Journal what happened while fresh. 4. Classify: normal variance (most likely — two -1.5% days/month are expected at 0.75% risk) vs execution errors vs edge problem. 5. Only edge-problem classification feeds the weekly review; variance days are filed and forgotten.

PB-2: Strategy benched by decay detector 1. Note the z-score path in the strategy's log. 2. Check regime context: favored regime absent (wait, size 0, keep monitoring) or regime present + still losing (edge death candidate). 3. Schedule re-validation in the monthly cycle. 4. Two failed re-validations — retire, archive reasons.

PB-3: Portfolio DD -12% (halving event) 1. Sizes halve automatically. 2. Operator re-reads \$3 and \$29. 3. Full report: which strategies contributed, live-vs-model drift, breaker history. 4. No new strategies, no re-mines deployed until DD < -6%. 5. Restore full size only after DD < -6% sustained 2 weeks.

PB-4: Portfolio DD -20% (full stop) 1. Everything flat, alerts disarmed. 2. Minimum 2-week cooling period — no trading, analysis only. 3. Full system audit: data integrity, cost realism, validation controls, journal adherence. 4. Written post-mortem before any restart. 5. Restart = back to paper for 4 weeks minimum, at half size.

PB-5: Exchange outage / API failure 1. Health monitor alerts. 2. Open positions: check via exchange app directly; if unmanageable, reduce exposure when possible. 3. Log outage window; affected trades flagged data_suspect in journal. 4. Resume only after data backfill passes validation.

PB-6: Data corruption detected 1. Validator quarantines affected pair-months. 2. Identify downstream runs using corrupt range → mark runs suspect in registry. 3. Re-download from archive, re-validate, re-run affected backtests. 4. Post-incident note in decision log.

PB-7: Discipline breach (traded without a card) 1. Trade journaled as violation (no hiding — the journal is for truth). 2. Third in a week → lockout engages. 3. Weekly review examines the trigger (boredom? revenge? FOMO?) and one countermeasure is adopted. 4. Pattern of breaches = size reduction until adherence $\geq 95\%$ for a month.

PB-8: Winning streak euphoria (the quiet killer) 1. If trailing 20-trade win rate > backtest 75th percentile → expect mean reversion, do NOT increase size. 2. Sizing-up law (\$30.1) still governs — streaks never override it. 3. Sweep profits to self-custody on schedule, not on feelings.

C2.8 Report Templates

C2.8.1 Daily digest (auto, Telegram)

APEX DAILY — 2026-08-07

Equity: \$1,042.30 (+0.6% today) | DD: -2.1%

Trades: 2 (both on-card ✓) — S1 ETH +0.31R, S3 BTC -1.0R

Breakers: none | Daily budget used: 0.9/1.5%

Decay watch: S5 z=-1.2 (watch), others healthy

Tomorrow's cards: EC-104 (ETH 07:00-16:00), EC-118 (BTC us session)

System: data ✓ jobs ✓ backup ✓

C2.8.2 Weekly report (auto + your notes)

APEX WEEKLY — W32 2026

PORTFOLIO: week +1.8% | month -0.4% | DD -2.1% (MC 61st pct)

PER STRATEGY (live vs backtest expectancy):

S1: live +0.16R (n=9) vs bt +0.21R — within band ✓

S3: live +0.08R (n=12) vs bt +0.19R — drift z=-1.1 WATCH

S5: live -0.05R (n=6) vs bt +0.22R — drift z=-1.8 SIZE HALVED

DISCIPLINE: 11/11 on-card (100%), avg emotion 2.1, 0 violations

RISK: VaR95 1.1% (limit 1.5%), correlation guard: no alerts

DATA: 0 gaps, freshness ✓, backup ✓

ACTIONS: S5 on watch → re-validate if z < -2 next week;

review Tuesday's S3 fill (slippage 3.1bps vs model 2)

C2.8.3 Monthly deep-dive (auto + 2h review)

Sections: (1) portfolio tear sheet (quantstats full metrics + MC band position); (2) per-strategy tear sheets + drift analysis; (3) allocation proposal (HRP weights, diff vs current, accept/decline

with reason); (4) re-validation queue results (cards renewed/retired); (5) miner funnel summary + new candidates entering validation; (6) cost calibration update (measured vs modeled slippage/fees); (7) data health + system incidents; (8) discipline review (adherence trend, violation themes); (9) capital decision (tier movement per \$30.1 — evidence attached); (10) next month's focus (one page, max three priorities).

C2.9 Journal & Edge Card Templates

C2.9.1 Pre-trade checklist (journal form, all required)

#	Check	Answer
1	Active Edge Card ID exists for this setup	EC-__
2	All card conditions currently true (session/day/regime/ADX...)	yes/no
3	No event blackout active	yes/no
4	Daily loss budget untouched, breakers clear	yes/no
5	Size taken from Risk Engine (not adjusted)	qty __
6	Stop price known BEFORE entry	price __
7	Emotion score 1–5 logged	__
8	Reason in one sentence (if it takes two, skip)	__

C2.9.2 Post-trade journal fields

Fill within 5 minutes of exit: actual fill vs plan (slippage), exit reason, rule compliance (Y/N + which rule if N), emotion at exit, lesson in ≤ 15 words, optional screenshot filename. Weekly: tag the best/worst trade and say why in one line each.

C2.9.3 Edge Card document (one page per card)

Header: ID, strategy, pair, timeframe, status, created/last-validated dates. Body: exact filter conditions; skip conditions; stats block (n, expectancy R, WR, PF, p_adj, both-halves stability, OOS stats); sizing rule reference; regime context (when it thrives/suffers); expiry date; lineage (mining run → validation run IDs).

C2.10 The 90-Day Condensed Quick-Start

For when you want the whole plan on one page. Gates are law; weeks are guidelines.

Days	Focus	Done when
1–7	Repo, env, config, CI	G0 green

Days	Focus	Done when
8–21	Data Lake: download, validate, resample, update	G1 (A1.1–A1.8)
22–35	Features + ONE strategy heartbeat: sweep → DB → first slice	G2 ★
36–49	Engine hardening + block library + Miner-1 first run	G3
50–63	Trade DB at scale + Edge Analytics + first cards	G4 (A6.3 green)
64–77	Validation Lab; run gauntlet; expect 2–8 survivors	G5 (A7.3 green)
78–84	Portfolio + Risk engines; breaker drills	G6
85–90	Command Center + alerts; begin paper trading	G7 → Phase 8

Daily minimum during build: 90 focused minutes. Weekly: one review hour. The plan tolerates slow weeks; it does not tolerate skipping gates.

C2.11 Code Skeletons for Core Components

These skeletons define the shape of the hardest components. They are deliberately incomplete (error handling and edge cases are yours to write), but they pin the architecture so you never stare at a blank file.

C2.11.1 Binance archive bulk downloader

```
# src/datalake/binance_archive.py
import io, zipfile, requests, pandas as pd
from pathlib import Path

BASE = "https://data.binance.vision/data/spot/monthly/klines"
COLS = ["open_time", "open", "high", "low", "close", "volume",

"close_time", "quote_vol", "trades", "taker_buy", "taker_buy_q", "ignore"]

def months_between(start: str, end: str) -> list[str]:
    return [str(p.date())[:7] for p in
            pd.period_range(start, end, freq="M")]

def fetch_month(pair: str, tf: str, ym: str) -> pd.DataFrame | None:
    url = f"{BASE}/{pair}/{tf}/{pair}-{tf}-{ym}.zip"
    r = requests.get(url, timeout=60)
    if r.status_code == 404:
```

```

        return None                                # pair not listed yet
    r.raise_for_status()
    with zipfile.ZipFile(io.BytesIO(r.content)) as z:
        df = pd.read_csv(z.open(z.namelist()[0]), header=None,
names=COLS)
        df["open_time"] = pd.to_datetime(df["open_time"], unit="ms",
utc=True)
        return df.drop(columns=["close_time", "taker_buy_q", "ignore"])

def ingest_pair(pair: str, tf: str, start: str, out_dir: Path):
    frames = []
    for ym in months_between(start,
pd.Timestamp.utcnow().strftime("%Y-%m")):
        if month_cached(out_dir, pair, tf, ym):    # resume-safe
            continue
        df = fetch_month(pair, tf, ym)
        if df is not None:
            cache_month(out_dir, pair, tf, ym, df)    # write per-month
fragment
        consolidate(out_dir, pair, tf)    # fragments → single parquet +
validate()

```

C2.11.2 Resampler (1m → all timeframes)

src/datalake/resample.py

```

AGG = {"open": "first", "high": "max", "low": "min", "close": "last",

"volume": "sum", "quote_vol": "sum", "trades": "sum", "taker_buy": "sum"}

def resample(df_1m: pd.DataFrame, tf: str) -> pd.DataFrame:
    rule = {"5m": "5min", "15m": "15min", "1h": "1h", "4h": "4h", "1d": "1D"}
    [tf]
    out = (df_1m.set_index("open_time")
            .resample(rule, origin="epoch")
            .agg(AGG)
            .dropna(subset=["open"]))    # gaps produce NaN rows →
drop
    return out.reset_index()
# Unit test: hand-built 120 one-minute bars → two 1h bars with known OHLC.

```

C2.11.3 TradeWriter with label derivation

src/tradesdb/writer.py

```

import duckdb, pandas as pd

```

```

def write_trades(con: duckdb.DuckDBPyConnection, run_id: str,
                 trades: pd.DataFrame, features: pd.DataFrame) -> int:
    df = trades.copy()
    df["run_id"] = run_id
    # --- derive edge labels by joining features at entry_time ---
    df = df.merge(features, left_on=["pair", "entry_time"],
                  right_on=["pair", "open_time"], how="left")
    df["hour_utc"] = df["entry_time"].dt.hour
    df["day_of_week"] = df["entry_time"].dt.dayofweek
    df["session"] = df["hour_utc"].map(session_of_hour)
    # trend_regime, vol_regime, rsi_at_entry, ... come from the merge
    before = con.execute("SELECT COUNT(*) FROM trades").fetchone()[0]
    con.register("df_stage", df)
    con.execute("""
        INSERT INTO trades BY NAME SELECT * FROM df_stage
        ON CONFLICT (trade_id) DO NOTHING
    """)
    after = con.execute("SELECT COUNT(*) FROM trades").fetchone()[0]
    return after - before          # idempotency test asserts re-write
== 0

```

C2.11.4 Slice statistics engine

src/edge/slice.py

```

import pandas as pd
from scipy import stats

```

```

def slice_stats(df: pd.DataFrame, min_n: int = 30) -> dict:
    n = len(df)
    if n < min_n:
        return {"n": n, "reliable": False}
    r = df["pnl_r"]
    wins, losses = r[r > 0], r[r <= 0]
    t_stat, p_val = stats.ttest_1samp(r, 0)
    return {
        "n": n, "reliable": n >= 100,
        "expectancy_r": r.mean(), "std_r": r.std(),
        "win_rate": (r > 0).mean(),
        "profit_factor": wins.sum() / max(-losses.sum(), 1e-12),
        "avg_win_r": wins.mean() if len(wins) else 0,
        "avg_loss_r": losses.mean() if len(losses) else 0,
        "max_consec_loss": max_consecutive(r <= 0),
        "t_stat": t_stat, "p_value": p_val,
        "ci95": bootstrap_ci(r.values, n_boot=5000),
    }

```

```

    }

def max_consecutive(loss_mask: pd.Series) -> int:
    groups = (~loss_mask).cumsum()
    return loss_mask.groupby(groups).sum().max() if len(loss_mask)
else 0

C2.11.5 Multiple-testing correction (Benjamini-Hochberg)
# src/edge/significance.py
import numpy as np, pandas as pd

def bh_adjust(scan_df: pd.DataFrame, q: float = 0.05) -> pd.DataFrame:
    df = scan_df.sort_values("p_value").reset_index(drop=True)
    m = len(df)
    df["bh_threshold"] = (df.index + 1) / m * q
    df["significant"] = df["p_value"] <= df["bh_threshold"]
    # enforce monotonicity: significance cut at last passing index
    if df["significant"].any():
        cut = df.index[df["significant"]].max()
        df["significant"] = df.index <= cut
    return df.sort_index()

C2.11.6 Monte Carlo battery
# src/validation/montecarlo.py
import numpy as np

def mc_resuffle(pnl_r: np.ndarray, sims: int = 10_000,
                rng: np.random.Generator = None) -> dict:
    rng = rng or np.random.default_rng()
    max_dds = np.empty(sims); finals = np.empty(sims)
    for i in range(sims):
        curve = np.cumsum(rng.permutation(pnl_r))
        peak = np.maximum.accumulate(curve)
        max_dds[i] = (peak - curve).max()
        finals[i] = curve[-1]
    return {"dd_p50": np.percentile(max_dds, 50),
            "dd_p95": np.percentile(max_dds, 95),
            "dd_p99": np.percentile(max_dds, 99),
            "final_p05": np.percentile(finals, 5),
            "sims": sims}

def mc_skip(pnl_r: np.ndarray, skip_rate: float, sims=10_000,
            rng=None):
    rng = rng or np.random.default_rng()

```

```

        keep = lambda: pnl_r[rng.random(len(pnl_r)) > skip_rate]
        exp = np.array([keep().mean() for _ in range(sims) if
len(keep())])
        return {"median_expectancy": np.median(exp),
                "p05_expectancy": np.percentile(exp, 5)}

def mc_block_bootstrap(pnl_r: np.ndarray, block: int = 20,
                        sims: int = 10_000, rng=None):
    rng = rng or np.random.default_rng()
    n_blocks = len(pnl_r) // block
    blocks = pnl_r[: n_blocks * block].reshape(n_blocks, block)
    dds = []
    for _ in range(sims):
        sample = blocks[rng.integers(0, n_blocks, n_blocks)].ravel()
        curve = np.cumsum(sample)
        dds.append((np.maximum.accumulate(curve) - curve).max())
    return {"dd_p95": np.percentile(dds, 95)}

```

C2.11.7 Walk-forward engine

src/validation/walkforward.py

import pandas **as** pd

```

def anchored_folds(start: str, end: str, train_years=3,
                    test_months=6) -> list[tuple]:
    folds, train_start = [], pd.Timestamp(start)
    end_ts = pd.Timestamp(end)
    while True:
        train_end = train_start + pd.DateOffset(years=train_years)
        test_end = train_end + pd.DateOffset(months=test_months)
        if test_end > end_ts:
            break
        folds.append((train_start, train_end, train_end, test_end))
        train_start += pd.DateOffset(months=test_months) # rolling

```

anchor

return folds

```

def walk_forward(strategy_cfg, folds, optimize_fn, evaluate_fn) ->
dict:
    oos_curves, metrics = [], []
    for (tr_s, tr_e, te_s, te_e) in folds:
        best = optimize_fn(strategy_cfg, tr_s, tr_e) # IS: pick
        plateau best
        m = evaluate_fn(best, te_s, te_e) # OOS:

```

untouched data

```
metrics.append(m); oos_curves.append(m["equity"])
is_ann = np.mean([m["is_ann_return"] for m in metrics])
oos_ann = np.mean([m["oos_ann_return"] for m in metrics])
return {"wfe": oos_ann / max(is_ann, 1e-9),
        "folds": metrics,
        "stitched_oos": pd.concat(oos_curves)}
```

C2.11.8 Edge-decay detector

src/monitoring/decay.py

```
import pandas as pd, numpy as np
```

```
def decay_z(live_trades: pd.DataFrame, bt_mean: float,
            bt_std: float, window: int = 30) -> dict:
    r = live_trades.sort_values("entry_time")["pnl_r"].tail(window)
    if len(r) < 10:
        return {"status": "insufficient", "n": len(r)}
    z = (r.mean() - bt_mean) / (bt_std / np.sqrt(len(r)) + 1e-12)
    status = ("healthy" if z > -1.5 else
              "watch"   if z > -2.0 else
              "halve"   if z > -2.5 else "bench")
    return {"status": status, "z": round(z, 2), "n": len(r),
            "roll_expectancy": round(r.mean(), 3)}
```

C2.11.9 HRP allocation wrapper (skfolio)

src/portfolio/allocate.py

```
import pandas as pd
from skfolio.preprocessing import prices_to_returns
from skfolio.optimization import HierarchicalRiskParity

def hrp_weights(strategy_returns: pd.DataFrame) -> pd.Series:
    model = HierarchicalRiskParity()
    model.fit(strategy_returns)
    w = pd.Series(model.weights_, index=strategy_returns.columns)
    return apply_constraints(w)      # $16.3 caps + correlation guard

def apply_constraints(w: pd.Series, cap=0.30) -> pd.Series:
    w = w.clip(upper=cap)
    return w / w.sum()              # renormalize after capping
```

C2.11.10 Telegram alert job

src/execution/alerts.py

```
import requests, os
```

```

TOKEN = os.environ["TELEGRAM_BOT_TOKEN"]      # never in git – env var
only
CHAT  = os.environ["TELEGRAM_CHAT_ID"]

def send(message: str) -> bool:
    r = requests.post(
        f"https://api.telegram.org/bot{TOKEN}/sendMessage",
        json={"chat_id": CHAT, "text": message}, timeout=10)
    return r.ok

def session_open_alert(card: dict, size: dict):
    send(f"    EDGE #{card['card_id']} ACTIVE – {card['strategy']} "
        f"{card['pair']} {card['timeframe']}\n"
        f"Conditions met. Size: {size['qty']} ({size['risk_pct']}%
risk)\n"
        f"Invalidation: {card['invalidation']}")

```

Skeleton governance: these files are the “shape contracts.” When your implementation diverges (it will), update the skeleton in this document in the same commit — the master plan and the code must never drift apart.

C2.12 Module Build Task Sequences (Step-by-Step)

Granular task lists for each module — check off tasks in order; each ends at the module’s QA checklist.

M1 Data Lake — 14 tasks

#	Task	Verify
1	Write fetch_month() for one pair/tf; inspect raw CSV	Columns match \$9.4
2	Add month-cache + resume logic	Kill mid-run, restart skips done months
3	Ingest BTCUSDT 1m full history	Row count vs Binance UI sample
4	Write validator v1 (gaps, dupes, OHLC rules)	Runs on BTC, produces report
5	Fix/remediate flagged months	Report clean
6	Loop all 20 pairs (background, days)	Coverage matrix filled
7	Write resampler + unit test	Hand-check passes
8	Generate 5m–1d for all pairs	Continuity check passes
9	DuckDB views v_bars_*	Query < 2s for 1y 15m

#	Task	Verify
10	get_bars() access API	Returns correct slice
11	Nightly updater via CCXT	Idempotent second run
12	Wire APScheduler job	Job log shows success
13	Event calendar CSV + loader	Events queryable
14	Quality report automation — Telegram	First report received

M2 Feature Factory — 11 tasks

#	Task
1	Indicator wrapper (pandas-ta) for momentum group
2	Trend + volatility + volume groups
3	Time/session label functions
4	Regime labelers (trend, vol) per \$10.3
5	Lookahead test harness (C2.6.1)
6	Synthetic fixtures (C2.6.2) for 5 core indicators
7	Materialize features.parquet per pair/tf
8	Feature versioning + rebuild script
9	Event join (minutes_to_event etc.)
10	Funding features (perp archive)
11	Nightly feature rebuild job

M3 Strategy Miner — 12 tasks

#	Task
1	Block base classes + registry
2	Implement 8 triggers + tests
3	Implement 8 filters + 5 exits + tests
4	Complete v1 block catalog (24/18/10)
5	vectorbt screen runner + cost config
6	Stage-1 solo scan all pairs/tfs
7	Stage-2 pair-up pipeline + chunking
8	TradeWriter integration (every run logs)
9	Stage-3 exit search
10	Plateau analyzer
11	Run registry + trial accounting

#	Task
12	Ranking + candidate handoff file

M4 Backtest Engine — 9 tasks

#	Task
1	Zero-cost parity test (C2.6.3)
2	Cost model (fees+slippage)
3	Next-bar fill enforcement
4	Intrabar conservative rule
5	Run-registry writer
6	Metrics panel + quantstats tear sheet
7	NautilusTrader wrapper for one strategy
8	Parity check vectorbt↔Nautilus
9	Reproducibility hash test

M5 Trade Database — 8 tasks

#	Task
1	Schema DDL + migration runner
2	TradeWriter + label joiner
3	Idempotent upsert
4	Analytical views (v_strategy_summary etc.)
5	Backtest vs live label parity
6	Performance tuning (partition if needed)
7	Backup script + schedule
8	Restore drill

M6 Edge Analytics — 9 tasks

#	Task
1	slice_stats engine (C2.11.4)
2	Dimension scanners (hour×day, session×regime)
3	Bootstrap CI implementation
4	BH correction (C2.11.5)
5	Both-halves stability check
6	Negative control (A6.3)
7	Positive control (A6.4)

#	Task
8	Edge Card generator + storage
9	Expiry job

M7 Validation Lab — 10 tasks

#	Task
1	Fold generator (anchored)
2	Walk-forward runner + WFE
3	MC reshuffle + skip + block bootstrap
4	Cost-stress rerun harness
5	Regime grid gate
6	PBO (CSCV) implementation
7	DSR implementation + trial input
8	Gauntlet orchestrator (gates in order)
9	Noise control (A7.3) — THE critical test
10	Positive control (A7.4)

M8 Portfolio — 6 tasks • M9 Risk — 8 tasks

#	Task
M8-1	Strategy return-series builder (stitched OOS)
M8-2	skfolio HRP wrapper + constraints
M8-3	Correlation guard
M8-4	Allocator backtest vs equal weight
M8-5	Monthly rebalance job + archive
M8-6	Tier logic (T1/T2/T3)
M9-1	Sizing engine (base × confidence × decay × weight)
M9-2	9 circuit breakers
M9-3	VaR/CVaR historical
M9-4	GARCH forecaster (arch)
M9-5	VaR backtest (Kupiec)
M9-6	Decay-multiplier wiring
M9-7	Risk metrics dashboard queries
M9-8	Breaker → Telegram alerts

M10 UI — 8 tasks • M11 Execution — 7 tasks • M12 Monitoring — 6 tasks

#	Task
M10-1	Streamlit skeleton + navigation
M10-2	Overview page
M10-3	Edge Explorer
M10-4	Validation Center
M10-5	Portfolio page
M10-6	Journal page + form
M10-7	Data Manager page
M10-8	Performance pass (caching)
M11-1	Edge Card publisher (daily job)
M11-2	Telegram alert bot
M11-3	Journal form + checklist enforcement
M11-4	Violation counter + lockout
M11-5	Testnet bridge (CCXT)
M11-6	Kill-switch
M11-7	Slippage calibration job
M12-1	Decay detector + z-score wiring
M12-2	Daily digest job
M12-3	Weekly report generator
M12-4	Monthly deep-dive generator
M12-5	Health monitor (freshness/jobs/disk)
M12-6	Event audit trail in DB

PART D — DATA & MATH REFERENCE

21. Complete Database Schemas (Consolidated)

Core tables were defined in Module 5. Supporting tables:

```
CREATE TABLE IF NOT EXISTS equity_curves (  
    run_id VARCHAR, ts TIMESTAMP, equity DOUBLE,  
    PRIMARY KEY (run_id, ts)  
);
```

```
CREATE TABLE IF NOT EXISTS allocations (  
    month DATE, strategy VARCHAR, weight DOUBLE,  
    method VARCHAR, created_at TIMESTAMP,  
    PRIMARY KEY (month, strategy, method)
```

```

);

CREATE TABLE IF NOT EXISTS breaker_events (
    event_id BIGINT PRIMARY KEY, ts TIMESTAMP,
    kind VARCHAR, --
    'daily_loss', 'weekly_loss', 'strategy_dd', 'portfolio_dd', 'lockout'
    detail VARCHAR, resolved_at TIMESTAMP
);

CREATE TABLE IF NOT EXISTS decay_events (
    event_id BIGINT PRIMARY KEY, ts TIMESTAMP,
    strategy VARCHAR, z_score DOUBLE, action VARCHAR, note VARCHAR
);

CREATE TABLE IF NOT EXISTS data_quality_log (
    log_id BIGINT PRIMARY KEY, ts TIMESTAMP, pair VARCHAR,
    timeframe VARCHAR, check_name VARCHAR, severity VARCHAR, detail
    VARCHAR
);

```

Partitioning note: DuckDB + Parquet happily scans millions of rows; if trades ever exceeds ~50M rows, export history to partitioned Parquet (trades/year=*/strategy=*) and query via views — same SQL, no migration pain.

22. Formula & Metrics Library

Expectancy (R): $E[R] = (WR \times \text{avg_win_R}) - ((1-WR) \times \text{avg_loss_R})$ — must be > 0 net of costs.

Profit factor: $PF = \text{gross_profit} / |\text{gross_loss}|$; ≥ 1.3 to survive screening, ≥ 1.15 OOS.

Sharpe (annualized): $SR = \text{mean}(r_daily) / \text{std}(r_daily) \times \sqrt{365}$ (crypto trades 365 days).

Sortino: downside deviation only; preferred for skewed crypto returns.

Calmar: $CAGR / |\text{maxDD}|$; ≥ 1.0 target at portfolio level.

Max drawdown: $\text{max_t} (\text{peak_t} - \text{equity_t}) / \text{peak_t}$ on the equity curve.

Walk-Forward Efficiency: $WFE = \text{ann_return_OOS} / \text{ann_return_IS}$; ≥ 0.5 .

Deflated Sharpe Ratio: $DSR = \Phi((\hat{SR} - E[\text{max SR under } H_0]) / SE(\hat{SR}))$ where $E[\text{max SR}]$ grows with $\sqrt{(2 \ln N)}$ for N variants tried, adjusted for skew/kurtosis — full formula implemented from Bailey & López de Prado (2014).

PBO: fraction of CSCV combinations where IS-optimal variant ranks below OOS median; logit transformation per López de Prado (2018).

Historical VaR_α: $\text{quantile}(r_{\text{daily}}, 1-\alpha)$; **CVaR_α:** mean of returns beyond VaR.

Parametric VaR (GARCH): $\text{VaR}_{95} = \hat{\mu} + \hat{\sigma}_{t+1} \times z_{0.05}$, $\hat{\sigma}$ from GARCH(1,1) via arch.

Position size: $\text{qty} = \text{equity} \times \text{risk\%} / \text{stop_distance\%} (\times \text{portfolio weight} \times \text{decay multiplier})$.

Kelly fraction: $f^* = \text{WR} - (1-\text{WR})/\text{payoff}$; deploy at most $f^*/4$.

Kupiec VaR backtest: $\text{LR} = -2 \ln[(1-p)^{(T-N)} p^N] + 2 \ln[(1-N/T)^{(T-N)} (N/T)^N] \sim \chi^2(1)$.

Multiple-testing corrections: Bonferroni α/m ; Benjamini–Hochberg FDR at $q=0.05$ (default for scans).

23. Forex Plug-In Specification (Future Market)

The system's DataSource interface makes forex a plug-in, estimated effort 1–2 weeks:

1. **Data:** Dukascopy tick archive (free, 2003+) via a downloader tool; resample ticks → 1m base bars. Expect slow bulk download (rate limits) — run over days, store once.
2. **Session labels:** replace crypto's UTC windows with asia/london/ny sessions + **DST handling** (US/EU transitions on different dates — use pytz history, never fixed offsets).
3. **Cost model:** variable spread by session (model from recorded spread series, not constant), swap/rollover for >1d holds (Wednesday triple), no taker fees (spread IS the fee).
4. **Universe:** EURUSD, GBPUSD, USDJPY, USDCHF, USDCAD, AUDUSD, NZDUSD + EURJPY, GBPJPY, EURGBP.
5. **Calendar:** high-impact events (NFP, FOMC, CPI, central-bank decisions) — mandatory for forex, strategies must trade around them.
6. **Validation delta:** same gauntlet; expect more mean-reversion edges in Asia session, momentum at London/NY opens.
7. **Execution:** MT5 demo → micro lots.

Everything downstream (miner, trade DB, edge analytics, risk, UI) works unchanged — that is the payoff of the interface discipline.

PART E — IMPLEMENTATION PLAN

24. Phased Build Plan (Week-by-Week)

Rules: phases are gated — you do not start a phase until the previous phase's exit gate checklist passes. Dates are indicative; the gates are law. Estimated total: **~12 weeks to paper trading**, ~5–6 months to first live trade, at ~10–15 focused hours/week.

Phase 0 — Foundations (Week 1)

Day	Task	Deliverable
1	Repo init, folder tree (\$8.1), git, pre-commit hooks	Clean repo
2	Environment, pinned requirements, sanity imports	.venv, requirements.txt
3	Config system (YAML loader + validation)	src/config.py
4	Logging + run-registry skeleton	src/runs.py
5	DuckDB bootstrap + migration runner	db/ tooling
6–7	CI: pytest scaffold, GitHub Actions (or local pre-push)	Green pipeline

Exit gate G0: repo builds clean on fresh clone; `pytest` runs (even if 3 tests); configs load; DuckDB initializes.

Phase 1 — Data Lake (Weeks 2–3)

- Week 2: bulk downloader + 1m ingestion for all 20 pairs (runs in background over days), validator, quality report
- Week 3: resampler + derived TFs, DuckDB views, nightly updater, gap remediation

Exit gate G1: Module-1 checklist A1.1–A1.8 fully evidenced.

Phase 2 — Feature Factory + Heartbeat Milestone (Weeks 4–5)

- Week 4: indicator layer + regime labels + session labels + lookahead tests
- Week 5: **hand-build ONE strategy** (recommended: session-window momentum breakout) → vectorbt sweep → TradeWriter → DuckDB → first edge-analysis query (hour×day heatmap)

Exit gate G2 (★ HEARTBEAT ★): end-to-end loop proven — bars → features → sweep → trades in DB → slice stats with significance. Module-2 checklist passes. *If this works, everything else is scaling.*

Phase 3 — Backtest Engine Hardening + Miner-1 (Weeks 6–8)

- Week 6: cost model, next-bar fills, intrabar-conservative rule, run registry; NautilusTrader wrapper for finalists
- Week 7: building-block library (20+ triggers, 15+ filters, 8 exits) with unit tests
- Week 8: staged mining pipeline (solo → pair-up → exit search → plateau) + first overnight 100k-variant run

Exit gate G3: Modules 3 & 4 checklists pass; first mining funnel report exists (e.g., 100,000 tested → 1,800 screened → 120 plateau-passing candidates).

Phase 4 — Trade DB at Scale + Edge Analytics (Weeks 9–10)

- Week 9: label pipeline hardening, views, backup routine
- Week 10: slice engine, dimension scans, significance layer (BH correction + stability halves), Edge Card generator + expiry

Exit gate G4: Module 5 & 6 checklists pass; **negative-control test A6.3 green** (random noise yields ~zero cards); first real Edge Cards generated from Phase-3 candidates.

Phase 5 — Validation Lab (Weeks 11–12)

- Walk-forward engine, MC battery ×4, regime stress, PBO, DSR
- Run the full gauntlet on Phase-4 candidates; produce kill list with reasons

Exit gate G5: Module-7 checklist incl. **A7.3 noise control (≥95% killed)**; expect 2–8 survivors from ~120 candidates. If 30+ “survive,” suspect a leakage bug — audit before proceeding.

Phase 6 — Portfolio + Risk Engines (Week 13)

- skfolio HRP wrapper + constraints, sizing engine, breakers, VaR/GARCH metrics

Exit gate G6: Modules 8 & 9 checklists pass; breaker simulation drills logged.

Phase 7 — Command Center (Week 14)

- All 7 Streamlit pages against real DB; performance budget; Telegram alerts

Exit gate G7: Module-10 checklist passes; you can operate the whole system from the UI.

Phase 8 — Paper Trading (Weeks 15–22, i.e., 2 months)

- Daily operation per runbook; every trade journaled; weekly drift reports; monthly calibration
- Parallel: slippage calibration, checklist-adherence tracking

Exit gate G8: ≥ 50 paper trades across ≥ 3 strategies; live expectancy within 1.5σ of backtest for majority of strategies; checklist adherence ≥ 95%; zero breaker overrides. **Only then live capital is discussed.**

Phase 9 — Live Micro-Capital + Miner-2 (Months 6–9)

- Live with money you can 100% afford to lose (Tier T1 sizing); genetic miner build-out; monthly re-mine loop operational

Phase 10 — ML Layer + Forex Plug-In (Months 9–12)

- Miner-3 (trade-success classifier as filter), Dukascopy forex plug-in, portfolio expands cross-market

The Master Funnel (what “healthy” looks like)

100,000+ variants mined

- 1,000–3,000 pass screening (~2%)
- 100–200 pass plateau analysis (~10%)
- 20–40 generate Edge Cards (~15%)
- 2–8 survive Validation Lab (~10–20%)

- 3-6 survive paper trading (~60%)
- 3-5 run live, sized by Risk Engine

25. Engineering Standards

25.1 Git discipline

- main = always-green; feature branches per module; PR-style self-review checklist before merge
- Commit messages: module: what (e.g., miner: add plateau scoring)
- Every research result must be traceable: run registry stores git_commit

25.2 Testing standards

Level	Scope	Tool	Coverage target
Unit	every indicator, block, labeler, sizer, breaker	pytest	≥ 80% on src/ core (datalake, tradesdb, risk, edge)
Property	no-lookahead, no-negative-weights, idempotent-writes	pytest + hypothesis	critical paths 100%
Integration	bar→trade→DB→slice chain	pytest fixtures	the heartbeat path always green
Statistical	negative/positive controls for miner, edge engine, validation lab	scripted sims	mandatory before trusting outputs

25.3 Code quality

- Type hints on public functions; ruff lint; mypy on core modules
- No magic numbers — config only; no prints — structured logging (rich + log files per run)
- Every module has a README.md in its folder: purpose, inputs, outputs, how to test

25.4 Decision log

docs/decisions.md — every non-obvious choice (e.g., “next-bar fills chosen over same-bar”) dated with rationale. Month-8 you will thank month-2 you.

26. Quality Assurance & Acceptance Test Plan

26.1 The four QA levels

1. **Module acceptance** — the per-module checklists (A1–A12) in Part C; evidence files stored in docs/qa/<module>/
2. **Integration QA** — heartbeat pipeline test runs in CI on every merge to main
3. **Statistical QA** — the negative/positive control simulations; rerun after ANY change to miner/edge/validation code

4. **Operational QA** — the weekly runbook audit (\$27): data freshness, backup test, breaker drill monthly

26.2 System-level acceptance tests (before first live trade)

Test ID	Test	Pass standard
SYS-1	Full rebuild from scratch on clean machine	System operational $\leq$ 2 days using docs alone
SYS-2	Reproducibility: re-run 5 historical runs	Identical metrics hashes
SYS-3	Data-wall audit	No code path loads OOS data into miner (grep + runtime test)
SYS-4	Cost realism: backtest vs paper slippage	Paper slippage $\leq$ 1.5 $\times$ modeled
SYS-5	Breaker drill	All 9 breakers fire in simulation; kill-switch drill signed
SYS-6	Failure injection: kill internet mid-update, corrupt a Parquet, stop DuckDB	System degrades with alerts, never silently
SYS-7	Performance: 500k-trade DB, UI < 5s/page, slice queries < 3s	Timing log
SYS-8	Security: API keys (testnet) in env vars, never in git	git grep clean
SYS-9	Backup restore drill	Full system restored from backup $\leq$ 4 hours
SYS-10	Documentation audit	Every module README + this plan current to $\leq$ 2 weeks

26.3 Definition of Done (any work item)

Code written $\rightarrow$ tests green $\rightarrow$ config externalized $\rightarrow$ logged $\rightarrow$ run registered $\rightarrow$ QA evidence filed $\rightarrow$ docs updated $\rightarrow$ checklist box ticked. **No tick without evidence.**

PART F — OPERATIONS

27. Daily / Weekly / Monthly Runbooks

27.1 Daily runbook (15–25 min)

Pre-session (5 min) - [] Open Command Center $\rightarrow$ Overview; confirm system health green (data fresh, no failed jobs) - [] Read today's Edge Cards; write down: strategy, pair, window,

size, invalidation - [] Check risk budget: daily loss remaining, open exposure, breakers armed -
[] Check event calendar — any blackout windows today?

During session - [] Trade ONLY on alerts/cards; complete journal checklist BEFORE entry; set stop immediately after entry - [] No improvising: no card = no trade; size from Risk Engine = size placed

End of day (5 min) - [] Journal complete for every trade (fill, emotion, screenshot optional) - [] Verify nightly jobs scheduled; glance at daily digest when it arrives

27.2 Weekly runbook (Sunday, 45–60 min)

- ☐ Review weekly report: per-strategy live-vs-backtest drift, decay z-scores
- ☐ Journal audit: adherence %, worst discipline lapse, one improvement for next week
- ☐ Data quality: gap report clean, freshness OK, backup ran
- ☐ Review any breaker/bench events — understand why
- ☐ Update strategy roster notes; plan next week's sessions

27.3 Monthly runbook (first Sunday, 2–3 hours)

- ☐ Monthly deep-dive report reviewed (tear sheets per strategy + portfolio)
- ☐ Portfolio rebalance: apply/decline proposal with written reason
- ☐ Re-validation queue: any Edge Card near 90-day expiry → re-validate or retire
- ☐ Miner: launch monthly re-mine on refreshed data; review survival funnel
- ☐ Slippage calibration: update cost config from live measurements
- ☐ Breaker drill + backup restore spot-check
- ☐ Capital review: tier check (T1/T2/T3), sizing parameters current
- ☐ Update this master plan if reality diverged

28. Strategy Lifecycle Management

DISCOVERED (miner output)

| screening + plateau



CANDIDATE → edge cards drafted (in-sample)

| Validation Lab gauntlet (6 gates)



reason

x killed → archived with

VALIDATED

| paper trading 8 weeks, $n \geq 30$



x drift beyond 1.5σ → back to

VALIDATED (halved confidence) or KILLED

LIVE (confidence 1.0, decay_mult 1.0)

| 100 live trades confirming



CORE (confidence 1.25) ← monthly re-validation keeps it here



- └ decay $z < -1.5 \rightarrow$ WATCH (size $\times 0.5$)
- └ decay $z < -2.5 \rightarrow$ BENCHED (size 0, must re-pass Lab to return)
- └ 2 failed re-validations $\rightarrow$ RETIRED (archived, feeds failure research)

Portfolio hygiene rules: max 8 strategies live simultaneously (attention is finite); every live strategy has a written one-page spec (logic, filters, risk, expected regime); any strategy you can't explain in 60 seconds doesn't trade.

29. Manual Execution & Psychology Protocol

The system's statistical edge is worthless if the operator leaks money through behavior. These rules are as binding as the code.

29.1 The ten operator laws

1. **No card, no trade.** Ever. The journal enforces; violations lock you out.
2. **Size comes from the Risk Engine.** Not from conviction, not from "this one looks great."
3. **The stop is placed before the profit is imagined.** No naked positions.
4. **Two consecutive losses – 30-minute walk.** Not optional.
5. **Daily loss limit hit – close the laptop.** The breaker does this; you respect it.
6. **Never widen a stop.** Reducing size out is allowed; adding to losers is banned.
7. **Journal emotion score honestly** (1=calm ... 5=euphoric/panicked). Trades at 4–5 get reviewed weekly — they correlate with mistakes.
8. **Boredom is not a signal.** Flat days are the system working.
9. **Drawdown protocol:** at portfolio -8% : sizes halved (automatic) + you re-read §3; at -12% : no new strategies added; at -20% : full stop, back to paper for 4 weeks. These are pre-commitments, not decisions to make in the moment.
10. **Weekly self-review is sacred.** 30 minutes, every Sunday, no exceptions.

29.2 Cognitive bias quick-reference (tape this near the screen)

Bias	Symptom	System countermeasure
Overconfidence after wins	Oversizing next trade	Fixed sizing, no discretion
Loss aversion	Widening stops, hoping	Law #6, stop placed at entry
Recency bias	Abandoning strategy after 3 losses	Decay detector needs $z < -2.5$ over 20+ trades
Hindsight bias	"I knew it" after the fact	Journal written BEFORE outcomes known
Gambler's fallacy	"We're due for a win"	Trade-by-trade independence; cards don't care
Action bias	Trading from boredom	No-card-no-trade lockout
Survivorship bias	Remembering winners only	The DB remembers everything — review it

30. Capital Plan, Costs & Risk Disclosures

30.1 Capital tiers and expectations

Phase	Capital at risk	Realistic goal
Build (months 1–3)	\$0	Skill + system
Paper (months 4–6)	\$0	Process proof
Micro live (months 6–9)	\$200–500	Fill/slippage reality, psychology under real stakes
Small live (months 9–18)	\$1k–5k	Confirm expectancy compounds; scale only on evidence
Growth (18m+)	Evidence-scaled	Add capital only from profits + proven track record

Sizing-up law: capital doubles only after (a) 100+ live trades, (b) expectancy positive and $\geq 60\%$ of backtest expectancy, (c) max DD within MC 95% band, (d) adherence $\geq 95\%$ for the trailing quarter.

30.2 Cost budget (stays \$0)

Data \$0 · APIs \$0 · compute = your machine · storage = local disk + free-tier cloud backup · exchange fees = trading costs (modeled, not overhead). The only money ever at risk is trading capital, and only after Phase 8.

30.3 Honest risk disclosures

- You can do everything right and still lose money for months; positive expectancy is a long-run property, not a weekly guarantee.
- Crypto carries exchange/custody risk: keep trading capital on exchange minimal; profits swept to self-custody monthly.
- Regulatory status of crypto varies by jurisdiction — know your local rules (taxes on every trade likely apply; the journal exports for tax reporting).
- Backtest realism has limits: flash events, exchange outages, and liquidity gaps exceed any model. Circuit breakers exist for the day the model is wrong.
- This system is educational infrastructure for your own trading; it is not financial advice and no outcome is guaranteed.

PART G — FUTURE & APPENDICES

31. Future Enhancements Roadmap

Ordered by value-per-effort; each is a module that plugs into the existing architecture.

31.1 Near-term (months 6–12)

Enhancement	Value	Effort
Miner-2 genetic programming	Discovers non-obvious logic humans wouldn't try	2–3 weeks
Miner-3 ML trade filter	Learns from your own trade DB; typically +10–20% expectancy on validated strategies	2 weeks
Regime-switching overlay	HMM (hmmlearn) market-regime detection → auto-enable/disable strategy families	1 week
Order-book features	Binance provides book snapshots; imbalance features for short-TF strategies	1–2 weeks
Auto-execution (Freqtrade)	Removes execution latency & emotion; only after S1–S2 gates	2 weeks
Funding-rate carry strategy family	Delta-neutral funding harvest (perp short + spot long) — a genuinely different return stream	2 weeks
Pairs/cointegration module	statsmodels cointegration on correlated alts; market-neutral reversion	1–2 weeks

31.2 Mid-term (year 2)

Enhancement	Notes
Forex plug-in (Chapter 23)	Second uncorrelated market; same system
React + FastAPI Command Center v2	Only when Streamlit genuinely limits you
Multi-timeframe ensemble strategies	HTF regime gates LTF entries systematically
Portfolio-of-portfolios	Meta-allocation across strategy families (trend / MR / carry / stat-arb)
Walk-forward automation	Monthly auto re-validation of the whole roster
Advanced execution	TWAP/VWAP slicing for larger size; limit-order fill modeling from book data
Hummingbot evaluation	If market-making ever interests you (different game, separate capital)

31.3 Long-term / research frontier

- **Alternative data:** on-chain metrics (free: exchange flows, active addresses) as regime features
- **LLM-assisted research (free-tier/local models):** code review of strategies, hypothesis brainstorming, report summarization — advisory only, never in the trading loop
- **Reinforcement learning position management:** research-grade only; deploy nothing without 12 months of evidence
- **Cross-exchange arbitrage monitoring:** educational; realistically closed to retail latency

31.4 Explicit non-goals (permanently)

- No paid data feeds, no paid APIs, no cloud bills in the core loop
- No high-frequency ambitions, no options market-making
- No strategy deployed that you cannot explain in 60 seconds

32. Scaling Path & Hardware Notes

Resource	Start	When it hurts	Fix
CPU	Any modern 4–6 cores	Miner-2 overnight runs	Multiprocessing pools; rent nothing
RAM	16 GB	vectorbt grids on 1m × 20 pairs	Chunk pairs; use 15m for screening
Disk	100 GB free	Tick data ambitions	Keep 1m as base; archive old runs
Backup	Second local disk	Data > 100 GB	Free-tier cloud (15 GB) for DB + configs only
Time	10–15 h/week	Operations > 1 h/day	Automate; the system should serve you, not employ you

Appendix A — Glossary

Alpha — return attributable to skill/edge rather than market exposure. **Backtest** — simulation of a strategy on historical data. **Circuit breaker** — pre-committed automatic risk limit. **CVaR (Expected Shortfall)** — average loss beyond the VaR threshold. **Deflated Sharpe Ratio** — Sharpe significance adjusted for multiple testing and non-normality. **Drawdown** — peak-to-trough equity decline. **Edge Card** — operational specification of when/where a validated strategy may trade. **Expectancy** — average profit per trade in R or bps, net of costs. **HRP** — Hierarchical Risk Parity allocation. **IC (Information Coefficient)** — correlation between signal and forward returns. **In-sample / Out-of-sample** — data used for discovery vs data reserved for testing. **MAE/MFE** — max adverse/favorable excursion during a trade. **PBO** — Probability of Backtest Overfitting (CSCV method). **Profit factor** — gross profit ÷ gross loss.

R-multiple — trade P&L expressed in units of initial risk. **Regime** — persistent market state (trend/volatility). **Slippage** — difference between expected and actual fill price. **Walk-forward analysis** — rolling optimize–test validation producing a stitched OOS curve. **WFE** — Walk-Forward Efficiency (OOS/IS return ratio).

Appendix B — Repository & Resource Directory

Resource	URL	Used for
Binance data archive	data.binance.vision	Bulk historical klines/trades
CCXT	github.com/ccxt/ccxt	Exchange connectivity
vectorbt	github.com/polakowo/vectorbt	Vectorized sweeps
NautilusTrader	github.com/nautechsystems/nautilus_trader	Event-driven backtest/live
backtesting.py	github.com/kernc/backtesting.py	Rapid prototyping
quantstats	github.com/ranaroussi/quantstats	Tear sheets
alphalens-reloaded	github.com/stefan-jansen/alphalens-reloaded	Factor IC analysis
skfolio	github.com/skfolio/skfolio	Portfolio optimization
Riskfolio-Lib	github.com/dcajasn/Riskfolio-Lib	Alt allocation methods
arch	github.com/bashtage/arch	GARCH/VaR
pandas-ta	github.com/twopirllc/pandas-ta	Indicators
DEAP	github.com/DEAP/deap	Genetic algorithms
gplearn	github.com/trevorstephens/gplearn	Genetic programming
Freqtrade	github.com/freqtrade/freqtrade	Live execution + FreqAI
DuckDB	github.com/duckdb/duckdb	Analytics database
Streamlit	github.com/streamlit/streamlit	Command Center
Dukascopy data	dukascopy.com/swiss/english/marketwatch/historical/	Forex plug-in

Appendix C — SQL Query Library (ready to use)

```
-- C1. Strategy league table (all sources)
SELECT strategy, COUNT(*) n, ROUND(AVG(pnl_r),3) exp_r,
       ROUND(SUM(pnl_r>0)*100.0/COUNT(*),1) wr,
```

```

        ROUND(SUM(pnl_r) FILTER(pnl_r>0)/NULLIF(-SUM(pnl_r)
FILTER(pnl_r<0),0),2) pf
FROM trades WHERE source='backtest'
GROUP BY 1 HAVING COUNT(*)>=100 ORDER BY exp_r DESC;

-- C2. Hour × day expectancy heatmap for a strategy
SELECT day_of_week, hour_utc, COUNT(*) n, ROUND(AVG(pnl_r),3) exp_r
FROM trades WHERE strategy=$1 AND source='backtest'
GROUP BY 1,2 HAVING COUNT(*)>=30 ORDER BY 1,2;

-- C3. Regime robustness grid
SELECT trend_regime, vol_regime, COUNT(*) n, ROUND(AVG(pnl_r),3) exp_r
FROM trades WHERE strategy=$1 GROUP BY 1,2 ORDER BY 1,2;

-- C4. Live vs backtest drift
SELECT strategy, source, COUNT(*) n, ROUND(AVG(pnl_r),3) exp_r,
        ROUND(STDDEV(pnl_r),3) sd
FROM trades WHERE strategy=$1 GROUP BY 1,2;

-- C5. Exit-reason autopsy (where does profit leak?)
SELECT exit_reason, COUNT(*) n, ROUND(AVG(pnl_r),3) exp_r,
        ROUND(AVG(mae_pct),2) avg_mae, ROUND(AVG(mfe_pct),2) avg_mfe
FROM trades WHERE strategy=$1 GROUP BY 1 ORDER BY exp_r;

-- C6. Consecutive-loss profile (psych prep + MC sanity)
WITH t AS (SELECT pnl_r, SUM(pnl_r>0) OVER w AS grp
        FROM trades WHERE strategy=$1 ORDER BY entry_time
        WINDOW w AS (ORDER BY entry_time))
SELECT COUNT(*) streak_len FROM (SELECT grp, SUM(pnl_r<=0) s FROM t
GROUP BY grp) z
WHERE s>0 GROUP BY streak_len ORDER BY 1;

```

Appendix D — Reading List (the theory behind the code)

1. **López de Prado** — *Advances in Financial Machine Learning* — PBO, purged CV, DSR, backtest overfitting. The Validation Lab's foundation.
2. **Bailey & López de Prado** — “The Deflated Sharpe Ratio” (paper, free) — Gate-6 math.
3. **Aronson** — *Evidence-Based Technical Analysis* — why data-mining bias destroys the naive; hypothesis-test thinking for traders.
4. **Chan** — *Algorithmic Trading* — practical strategy archetypes, mean-reversion & momentum families.
5. **Carver** — *Systematic Trading* — position sizing, vol targeting, combining subsystems (Risk & Portfolio Engines' philosophy).

6. **Grinold & Kahn — Active Portfolio Management** — IC, breadth, the fundamental law ($\text{edge} \times \sqrt{\text{breadth}}$).
7. **Taleb — Fooled by Randomness** — respect for noise; required annual re-read.
8. **skfolio & vectorbt docs** — treat as textbooks; read the method docstrings before using.

Appendix E — Master Project Sign-Off Checklist

The project is “operational” (ready for paper trading) only when ALL of these hold:

- ☐ G0–G7 phase gates evidenced and archived in docs/qa/
- ☐ Module checklists A1–A12: 100% ticked with evidence files
- ☐ Statistical controls: miner negative control, edge-engine false-positive control, validation-lab noise control — all green within the last 30 days
- ☐ ≥ 2 strategies VALIDATED (6/6 gates) with active Edge Cards
- ☐ Risk Engine: all 9 breakers drill-tested; kill-switch drill signed
- ☐ Backup restore drill completed ≤ 4 hours
- ☐ Command Center operating all 7 pages on live data
- ☐ Runbooks printed/accessible; operator laws taped to the desk
- ☐ This master plan reviewed and updated to reality

And “live-ready” adds: - ☐ Phase-8 gate G8 passed (50+ paper trades, drift within 1.5σ , adherence $\geq 95\%$) - ☐ Slippage calibration $\leq 1.5 \times \text{model}$ - ☐ Capital at risk is money you can lose without changing your life - ☐ One-page spec per live strategy written and understood

Appendix F — Common Pitfalls & Troubleshooting

Symptom	Likely cause	Fix
Amazing backtest, dead live	Same-bar fills, lookahead, or costs off	Re-audit Gates A4.3/A2.1; slippage calibration
Miner “finds” 500 great strategies	Multiple-testing illusion OR leakage	Check trial accounting — DSR; run negative control A7.3
Slice looks great, $n=23$	Small-sample mirage	Provisional status; wait or widen slice
Strategy great until 2024, dead after	Edge decay or regime shift	Decay detector verdict; re-mine; retire gracefully
Equity curve too smooth	Intrabar optimistic assumption	Enforce conservative intrabar rule (A4.4)
DuckDB slow	Row-wise inserts, no views	Batch inserts; pre-aggregate views; partition Parquet
vectorbt memory blow-up	Grid $\times 1m \times 20$ pairs at once	Chunk pairs/timeframes; screen on 15m first
“It worked in vectorbt but not Nautilus”	Fill/timing difference	Parity test A4.5; find the exact divergent trade
Constantly tweaking the	Action bias applied to	Changes only via weekly

Symptom	Likely cause	Fix
system	engineering	review; decision log
Skipping journal entries	Discipline leak	Lockout rule; adherence in weekly audit
One pair dominates P&L	Pair-specific luck	Require cross-pair robustness before scaling
Drawdown feels unbearable	Size too big for your psychology	Halve base risk; the best size is one you can hold

Appendix G — Worked Mathematics & Sizing Workbook

Numeric examples you'll reuse constantly. Verify your implementation against these by hand.

G.1 Expectancy arithmetic

Strategy: WR 44%, avg win +1.9R, avg loss -1.0R, round-trip cost 0.06R.

$$E[R] = 0.44 \times 1.9 - 0.56 \times 1.0 - 0.06 = 0.836 - 0.56 - 0.06 = +0.216R/\text{trade}$$

At 0.75% risk/trade and 60 trades/month: $E[\text{monthly}] \approx 60 \times 0.216 \times 0.75\% \approx +9.7\%/\text{month}$

At 0.75% risk/trade and 8 trades/month: $E[\text{monthly}] \approx +1.3\%/\text{month}$

Lesson: expectancy $\times$ frequency $\times$ size = income. Filters trade frequency for expectancy — the edge analysis exists to optimize that product, not any single term.

G.2 Position sizing (three examples)

Equity \$4,000; risk 0.75% — \$30 at risk. - **Ex 1:** ETHUSDT, entry 3,200, stop 3,168 (1% below). $\text{qty} = 30 / 0.01 = \$3,000$ notional ≈ 0.937 ETH. Portfolio weight 0.5 (if applied) — 0.469 ETH. - **Ex 2:** BTCUSDT, stop distance 0.4%. $\text{qty} = 30 / 0.004 = \$7,500$ notional. Same risk, bigger size — stop distance sets size, never the reverse. - **Ex 3:** decay_mult 0.5 active — \$15 at risk — halve whichever notional the formula gives.

G.3 Kelly criterion sanity check

WR 45%, payoff 1.8. $f^* = 0.45 - 0.55/1.8 = 0.144$ — quarter-Kelly = 3.6% of equity per trade — far above our 0.75% fixed-fractional. Conclusion: fixed-fractional is the conservative governor; Kelly is only ever a ceiling cross-check, never the sizing engine at our sample sizes.

G.4 Monte Carlo interpretation

10,000 reshuffles of a 600-trade sequence with maxDD realized 11%:

MC maxDD percentiles: 50th 12% | 75th 15% | 90th 18% | 95th 21% | 99th 27%

Reading: realized 11% sits at the ~40th percentile — strategy behaved as expected. If live DD reaches 19%, that's the ~93rd percentile: not proof of death, but decay detector should already be halving size. The MC band converts “drawdown panic” into a percentile reading.

G.5 VaR & CVaR by hand

Daily portfolio returns over 500 days, sorted worst 25 (= 5%): -3.2% ... -1.9%. - Historical VaR95 $\approx$ 1.9% (the 25th-worst day boundary) - CVaR95 $\approx$ mean of worst 25 $\approx$ -2.5% - GARCH parametric: $\hat{\sigma}(t+1) = 1.15\%$ $\rightarrow$ VaR95 = $1.645 \times 1.15\% \approx 1.89\%$ (consistent $\checkmark$) - Kupiec check: expected 25 exceptions, observed 28 $\rightarrow$ within acceptance region (don't reject model).

G.6 Deflated Sharpe intuition

Same strategy, same Sharpe 1.4, two contexts: - Tried 10 variants $\rightarrow$ $E[\max \text{SR} \mid H_0] \approx 0.8$ $\rightarrow$ DSR comfortably positive $\rightarrow$ PASS - Tried 100,000 variants $\rightarrow$ $E[\max \text{SR} \mid H_0] \approx 1.5$ $\rightarrow$ DSR ≈ 0 $\rightarrow$ FAIL Identical performance, opposite verdicts — because luck's expected maximum grows with trials. This is why the miner's trial accounting (n_variants) is sacred.

G.7 Risk-of-ruin framing

With $E[R] = +0.2$, $\sigma(R) = 1.2$, risk 0.75%/trade: risk of hitting the -20% stop before doubling is small (single digits %). At 3% risk/trade it multiplies several-fold. The 0.5–1% band isn't timidity; it's what keeps the -20% event rare enough that PB-4 is a procedure you'll likely never run.

Appendix H — Failure-Mode Library (25 Ways Quant Systems Die)

Study this list quarterly. Each entry: the failure, why it happens, and the system's built-in defense.

#	Failure mode	Mechanism	APEX defense
1	Lookahead bias	Feature uses future data	A2.1 lookahead tests on everything
2	Same-bar fills	Signal and fill on same bar	next-bar-open rule, A4.3 test
3	Intrabar optimism	TP assumed before SL in one bar	conservative rule, A4.4
4	Cost amnesia	Backtest without fees/slippage	costs mandatory in config; A4.2
5	Spread blindness	Fixed spread assumption	measured calibration loop \$19.3
6	Survivorship in universe	Only today's surviving pairs	include delisted pairs in archive pulls
7	Multiple-testing illusion	100k trials $\rightarrow$ fake edges	BH correction, PBO, DSR, trial accounting
8	Peak-picking	Choosing best	plateau scoring \$11.3

#	Failure mode	Mechanism	APEX defense
9	Small-sample trust	backtest params n=30 “edge”	Stage 4 n≥100 rule, provisional tier
10	Regime overfit	Works only in 2021 bull	regime stress gate; both-halves stability
11	Walk-forward theater	WF run once, cherry- picked	full anchored folds, WFE threshold
12	OOS peeking	“Just checking” 2024 results	data wall, SYS-3 audit
13	Parameter creep	Tuning after seeing OOS	new OOS required; decision log
14	Correlation pile-up	5 strategies = 1 bet	correlation guard, HRP clusters
15	Sizing by conviction	Bigger on “sure things”	Risk Engine only; journal violations
16	Breaker override	“Just this once”	code-enforced lockouts, PB-1
17	Revenge trading	Chasing losses	lockouts, operator law #4
18	Edge decay denial	Riding dead strategy	decay detector, auto- bench
19	Rebalancing churn	Weekly weight fiddling	monthly cadence, turnover penalty
20	Complexity worship	Adding ML to broken basics	simple-first principle; ML-as- filter only
21	Data rot	Silent gaps/bad bars	validator, quarantine, PB-6
22	Backup theater	Backups never tested	restore drills SYS-9
23	Single-exchange risk	Exchange freezes	minimal balances, monthly sweeps
24	Journaling decay	Stops journaling → blind	adherence metric, lockout rule
25	Abandoning in drawdown	Quitting at the percentile bottom	MC percentile framing, PB-3/4

Appendix I — Complete Configuration Reference

I.1 config/system.yaml (annotated)

paths:

data: data/ *# data lake root (Parquet)*

```

db: db/apex.duckdb      # single-file analytics DB
runs: runs/              # run registry root
time:
  insample_start: "2017-08-17"  # Binance BTCUSDT listing
  insample_end:   "2022-12-31"  # ← THE RESEARCH WALL
  oos_start:      "2023-01-01"
random_seed: 42          # global default; miners get derived
seeds
execution_defaults:
  timezone: UTC          # system-wide; never local time

```

I.2 config/risk.yaml (annotated)

```

sizing:
  base_risk_pct: 0.75      # of current equity, per trade
  min_risk_pct: 0.50       # floor when confidence reduced
  max_risk_pct: 1.00       # hard ceiling, no exceptions
  confidence_multipliers: {provisional: 0.75, validated: 1.0, core:
1.25}
  vol_target_ann_pct: 12   # Phase-2 volatility targeting
  kelly_fraction_cap: 0.25 # quarter-Kelly ceiling cross-check
breakers:
  daily_loss_pct: 1.5
  weekly_loss_pct: 4.0
  strategy_dd_pct: 15
  portfolio_dd_halve_pct: 12
  portfolio_dd_stop_pct: 20
  max_concurrent_positions: 5
  max_same_direction_per_asset: 2
decay:
  watch_z: -1.5           # trailing 30-trade expectancy z-
score
  half_z: -2.0
  bench_z: -2.5
  bench_confirm_trades: 20

```

I.3 config/miner.yaml (annotated)

```

stage1: {timeframes: [5m, 15m, 1h, 4h], min_trades: 100}
stage2:
  top_trigger_pct: 30      # survivors of solo scan
  screen: {min_expectancy_r: 0.0, min_pf: 1.15, min_trades: 100}
stage3:
  top_combos: 500
plateau:

```

```

    perturbation_pct: 20
    min_neighbor_score: 0.6
ranking:
    weights: {expectancy_bps: 1.0, log_n: 1.0, plateau: 1.0,
consistency: 1.0}
budget:
    max_variants_per_run: 250000    # trial accounting feeds DSR
    parallel_workers: 8

```

I.4 config/validation.yaml (annotated)

```

walkforward: {train_years: 3, test_months: 6, min_folds: 6, min_wfe:
0.5,
                max_fold_dd_multiple: 1.5}
montecarlo: {sims: 10000, dd_budget_pct: 20, skip_rates: [0.1, 0.2,
0.3],
                block_len: 20, cost_stress: [2.0, 3.0]}
regime_grid: {min_cell_trades: 30, min_cell_expectancy_r: -0.3}
pbo: {blocks: 16, threshold: 0.20}
dsr: {alpha: 0.05}
edge:
    min_trades_slice: 100
    provisional_min: 50
    correction: bh                                # Benjamini-Hochberg FDR
    fdr_q: 0.05
    stability_halves: true
    card_expiry_days: 90

```

I.5 config/portfolio.yaml — see §16.3; config/universe.yaml — see §9.6;
config/backtest.yaml — see §12.3.

Appendix J — Extended SQL Query Library

-- C7. Mining funnel for a run

```

SELECT status, COUNT(*) FROM runs
WHERE run_id LIKE $1 || '%' GROUP BY 1;

```

-- C8. Strategy equity curve from trades (running sum)

```

SELECT entry_time::DATE d, SUM(pnl_quote) pnl,
       SUM(SUM(pnl_quote)) OVER (ORDER BY entry_time::DATE) cum
FROM trades WHERE strategy=$1 AND source='live'
GROUP BY 1 ORDER BY 1;

```

-- C9. MFE/MAE edge diagnostic (are stops/targets placed well?)

```

SELECT strategy,
       ROUND(AVG(mfe_pct),2) avg_mfe, ROUND(AVG(mae_pct),2) avg_mae,

```

```

        ROUND(AVG(pnl_pct),2) avg_pnl,
        ROUND(CORR(mfe_pct, pnl_pct),2) mfe_pnl_corr
FROM trades WHERE strategy=$1 GROUP BY 1;
-- Reading: high avg_mfe with low avg_pnl → targets too far;
-- low |mae| vs stop distance → stop too wide (wasted risk).

-- C10. Session × regime expectancy matrix
SELECT session, trend_regime, COUNT(*) n, ROUND(AVG(pnl_r),3) e
FROM trades WHERE strategy=$1 AND source='backtest'
GROUP BY 1,2 HAVING COUNT(*)>=30 ORDER BY e DESC;

-- C11. Monthly seasonality check (usually noise – verify before
believing)
SELECT month, COUNT(*) n, ROUND(AVG(pnl_r),3) e
FROM trades WHERE strategy=$1 GROUP BY 1 HAVING COUNT(*)>=50 ORDER BY
1;

-- C12. Live slippage calibration (journal vs plan)
SELECT strategy, COUNT(*) n,
        ROUND(AVG(actual_slip_bps),2) measured,
        ROUND(AVG(modeled_slip_bps),2) modeled,
        ROUND(AVG(actual_slip_bps)/AVG(modeled_slip_bps),2) ratio
FROM live_journal jl JOIN trades t USING(trade_id)
GROUP BY 1;

-- C13. Discipline audit
SELECT strftime(created_at,'%Y-W%V') wk, COUNT(*) trades,
        SUM(NOT checklist_ok) violations, ROUND(AVG(emotion_score),1)
emo
FROM live_journal GROUP BY 1 ORDER BY 1 DESC LIMIT 12;

-- C14. Edge-card health board
SELECT card_id, strategy, pair, status, n_trades, expectancy_r,
        last_validated, last_validated + INTERVAL 90 DAY AS expires
FROM edge_cards ORDER BY status, expires;

-- C15. Decay detector input (trailing 30-trade expectancy)
SELECT strategy, trade_id, pnl_r,
        AVG(pnl_r) OVER (PARTITION BY strategy ORDER BY entry_time
        ROWS BETWEEN 29 PRECEDING AND CURRENT ROW)
roll_exp
FROM trades WHERE source='live' ORDER BY strategy, entry_time;

```

PART H — TRADER'S REFERENCE LIBRARY

33. Statistics Primer for Traders

The minimum statistics you must understand to operate this system honestly. Each concept is stated, then translated into trading terms.

33.1 Distributions. Trade P&L is a random draw from the strategy's return distribution. Crypto trade returns are fat-tailed (extreme moves more common than a normal distribution predicts) and often skewed (trend systems: many small losses, few large wins — negative skew of losses, positive skew of P&L tail). Consequence: never use normal-distribution assumptions for risk; use historical/bootstrapped distributions (which is exactly why our Monte Carlo reshuffles real trades instead of simulating Gaussians).

33.2 Sample size and standard error. The standard error of a mean shrinks as $1/\sqrt{n}$. A slice of 25 trades with expectancy $+0.3R$ and $\sigma(R)=1.2$ has $SE \approx 0.24$ — the “edge” is 1.25 standard errors from zero, i.e., indistinguishable from luck. At $n=100$ the same numbers give $SE \approx 0.12$ (2.5σ — worth attention); at $n=400$, 5σ (real). This single formula — $z = E / (\sigma/\sqrt{n})$ — is the mathematical spine of the $n \geq 100$ rule, the decay detector, and the live-vs-backtest drift test.

33.3 Hypothesis testing. Null hypothesis H_0 : “this slice has no edge (true mean = 0).” The p-value is $P(\text{data this extreme} \mid H_0 \text{ true})$ — NOT the probability the edge is real. $p < 0.05$ means: if there were no edge, data like this would appear only 5% of the time. With one test, that's persuasive. With 500 tests (one hour×day scan), you EXPECT ~25 false positives at $p < 0.05$ — which is why §33.5 exists.

33.4 Confidence intervals. “Expectancy $+0.21R$, 95% CI $[+0.07, +0.35]$ ” beats “expectancy $+0.21R$ ” every time. The CI tells you the plausible range; if it crosses zero, the estimate is noise-grade regardless of how good the point looks. Bootstrap CIs (resample the trades 5,000×, take percentiles) handle the fat tails correctly and are built into the slice engine.

33.5 Multiple comparisons and FDR. Every additional hypothesis tested inflates the false-positive rate. Corrections: Bonferroni (divide α by number of tests — simple, conservative) and Benjamini–Hochberg (controls the expected fraction of false discoveries — more powerful, our default). The edge engine never reports a raw p-value without its correction context. Rule of thumb: if you scanned 168 hour×day cells, treat any unadjusted $p < 0.05$ as meaningless; only BH-passing slices may become cards.

33.6 Regression to the mean. Extreme observed performance (the best variant of 100,000) is almost always part skill, part luck — and the luck part evaporates forward. Expect OOS performance to be worse than IS, systematically (that's why $WFE \geq 0.5$ passes but $WFE = 1.0$ should make you suspicious of a leak).

33.7 Stationarity. Statistics assumes the distribution stays stable. Markets are non-stationary — vol clusters, regimes shift, edges die. This is THE fundamental problem of quantitative trading, and the system's entire monitoring layer (decay detection, monthly re-validation, card expiry) is the engineered response to it.

33.8 Independence and autocorrelation. Trades cluster in time; a losing streak is not 10 independent losses but often one bad regime. Block bootstrap (MC #3) exists because reshuffle assumes independence; when the two disagree, trust the block version.

33.9 Bayesian intuition (no math required). Prior: most strategy ideas fail (>90%). Evidence: a good backtest. Posterior: still probably fails — the validation gauntlet is how you accumulate enough evidence to update a skeptical prior. Operators who internalize this stop falling in love with backtests.

33.10 The two errors. Type I: trading a strategy that doesn't work (costs money). Type II: discarding one that does (costs opportunity). Retail optimizes against Type II (afraid to miss anything); professionals optimize against Type I (afraid to trade noise). This system is deliberately Type-I-averse: capital preservation beats missed chances, always.

34. Crypto Market Microstructure Primer

What you must know about the terrain you're trading.

34.1 Perpetuals vs spot. Perps track spot via the funding mechanism: when perp trades above spot, longs pay shorts (positive funding) and vice versa. Funding is a direct sentiment gauge (extreme = crowded positioning → S5 fade strategy) and a real cash flow if you hold perp positions. Spot has no funding but also no short option on most retail venues — most short strategies in this system assume perp execution.

34.2 Liquidity cycles. Despite 24/7 trading, volume follows a daily wave: deepest during the Europe-US overlap (roughly 12:00–16:00 UTC), thinnest in late-Asia hours and weekends. Consequences: (a) slippage varies 2–3× across the day — the cost model's session-varying slippage is not optional; (b) breakout strategies work best when participation arrives (session opens); (c) weekend price action is lower-quality — always test weekend exclusion as a filter.

34.3 Volatility regimes. Crypto vol clusters hard: quiet weeks of 1% daily ranges punctuated by 10% days. ATR-percentile regime labels capture this. Mean-reversion strategies harvest the quiet; momentum strategies harvest the storms; a strategy blind to vol regime is a coin flip on which world it's in.

34.4 Event structure. Macro events (FOMC, US CPI, payrolls) hit crypto sharply — the 30 minutes after release are spread-widening, wick-heavy chaos. Trade them only via a dedicated event strategy (S6) or stand aside (F08 blackout). Token-specific events (unlocks, mainnet launches, exchange listings) matter for alts.

34.5 Cross-pair correlation. Alts are highly correlated to BTC (typical 30d correlation 0.6–0.9). Five alt longs ≈ one big BTC beta position — the correlation guard and max same-direction limits exist for exactly this. It also means multi-pair “diversification” is partly illusory; true diversification comes from strategy family diversity (trend vs MR vs carry).

34.6 Exchange microstructure. Binance spot taker fees ~10bps (less with BNB discount/volume tiers — recalibrate config to YOUR actual tier), perp taker ~4–5bps. Market orders cross the spread; limit orders earn it but risk non-fill. In backtests, always pay taker costs unless the strategy is explicitly limit-based with a fill model — optimism here is the classic way backtests lie.

34.7 Weekend and holiday effects. Institutional participation drops on weekends and US holidays; ranges compress but stop-hunts in thin books produce vicious wicks. Test, don't assume: some pairs show real weekend edges; most show noise with worse fills.

34.8 Stablecoin basis and risk. P&L in USDT is P&L in a dollar-IOU. Operationally: keep balances modest, sweep profits, and remember depeg tail-risk exists. The system's custody rule (monthly sweeps) is non-negotiable.

35. KPI Definitions & Dashboard Metrics

The numbers that define system health, with exact definitions so nothing is ever eyeballed.

KPI	Definition	Green	Yellow	Red
Live expectancy drift z	(live rolling 30-trade mean – backtest mean) / SE	> -1.5	-1.5...-2.5	< -2.5
Checklist adherence	% of trades with complete pre-trade checklist	≥ 95%	85–95%	< 85%
On-card rate	% of trades with active Edge Card	100%	95–99%	< 95%
Cost ratio	measured live slippage ÷ modeled	≤ 1.2	1.2–1.5	> 1.5 → recalibrate
Data freshness	age of newest bar at morning check	< 12h	12–24h	> 24h
Backup age	days since verified backup	≤ 7	8–14	> 14
Portfolio DD percentile	current DD vs MC distribution	< 75th	75–95th	> 95th → PB-3/4
VaR95 utilization	current VaR95 ÷ limit	< 70%	70–100%	> 100%
Strategy breadth	# live validated strategies	3–8	2 or 9–10	1 or >10
Correlation guard	max pairwise 90d strategy corr	< 0.5	0.5–0.6	> 0.6 → action
Validation backlog	candidates awaiting gauntlet	< 10	10–20	> 20 (schedule lab time)
Card freshness	% cards validated within	100%	90–99%	< 90%

KPI	Definition	Green	Yellow	Red
Miner cadence	90 days days since last mining run	≤ 30	31–45	> 45
Journal completeness	% trades with post-trade entry	100%	95–99%	$< 95\%$
Equity vs plan	actual equity vs MC median path	band $\pm 1\sigma$	$\pm 1-2\sigma$	outside 2σ either way

Review the full KPI board monthly; the red rows demand a written action in the monthly review.

36. First-Year Operating Calendar

Month	System focus	Trading focus	Milestone
M1	Phases 0–1: repo, data lake	None	G0, G1 green
M2	Phases 2–3: features, heartbeat, miner	None	G2 ★ heartbeat, G3
M3	Phases 4–5: edge analytics, validation lab	None	First validated strategies
M4	Phases 6–7: risk, UI; paper trading begins	Paper only	G6, G7; first paper cards
M5	Paper trading; Miner-2 build starts	Paper	25+ paper trades
M6	Paper trading; calibration loops live	Paper	G8 gate attempt (50+ trades)
M7	Micro-live (T1); monthly re-mine operational	Micro live	First real fills journaled
M8	Live ops; decay detector proving itself	Micro live	30+ live trades
M9	Miner-2 genetic runs feeding lab	Micro live	First GP-discovered candidate
M10	Miner-3 ML filter research	Small live if evidence	60+ live trades
M11	Forex plug-in (Dukascopy) build	Small live	Second market in data lake
M12	Annual review: full re-validation of everything	Small live	Year-1 report; year-2 plan

The annual review (M12) is a full-day ritual: every strategy re-validated on complete data, every assumption re-checked, this document revised to v2.0.

37. The Learning Track

Skills in the order the build needs them. 30–45 min/day alongside building; free resources only.

Weeks	Topic	Why now	Free resources
1–3	Python + pandas fluency	Everything is DataFrames	pandas docs, Kaggle pandas course
4–6	SQL + DuckDB	Edge mining is SQL	DuckDB docs, Mode SQL tutorial
5–8	Statistics core (Ch. 33 topics)	Validation lab needs it	StatQuest (YouTube), <i>Think Stats</i> (free PDF)
7–10	vectorbt + backtesting internals	Engine work	vectorbt docs & examples
9–12	López de Prado selected chapters	PBO/DSR/CV correctness	AFML book + author's free papers
13–16	Portfolio theory lite (HRP, risk parity)	Portfolio engine	skfolio docs + HRP original paper (free)
17–20	GARCH & VaR (arch docs)	Risk engine	arch docs, RiskMetrics doc (free)
21+	Market microstructure + trader psychology	Live phase	<i>Trading in the Zone</i> (Douglas), Ch. 34

38. Self-Interrogation FAQ

The questions you'll secretly ask yourself at 2 AM, answered in advance.

“The backtest is incredible — why shouldn't I just trade it now?” Because an unvalidated backtest is a hypothesis, not a result. Run the gauntlet. If it's real, it will still be real in two weeks.

“The miner found nothing this month — is it broken?” Check the controls (A7.3/A7.4). If controls pass, nothing-found is a legitimate market state. Mining dry spells are normal; forcing output is how noise gets promoted.

“A strategy passed everything but feels wrong.” Write down why. If the reason is mechanical (“I can't explain the logic”), it doesn't trade — the 60-second rule. If it's vibes, trade it small; feelings aren't evidence either.

“Three losing weeks — is the edge dead?” Compute the z-score (the dashboard does). A –1.5% week happens at 0.75% risk with no edge problems at all. The detector tells you when it's statistically meaningful; until then, variance is the boring answer.

“Can I just add one discretionary trade?” That’s violation #1 toward lockout. The system exists because discretionary-you underperforms systematic-you. Journal the urge instead — urges logged weekly are useful psychology data.

“Why not just buy a commercial tool (StrategyQuant etc.)?” Budget is zero, and black-box mining without the validation literacy this build gives you would lose money anyway. You’re building the tool AND the operator.

“When do I scale up?” \$30.1’s sizing-up law. Never earlier, never because of a hot streak, never because of impatience.

“What if everything fails after a year?” Then you’ll have something worth more than a strategy: a verified negative result set, institutional-grade process skills, and a database of 100k+ analyzed trades — assets that transfer to any future trading, quant job applications, or fund-raising credibility. The only true failure mode is skipping the process.

39. Universe & Correlation Management

The 20-pair universe is a managed portfolio in itself, not a fixed list.

39.1 Admission rules. A pair enters the research universe when: (a) ≥ 2 years of history on Binance; (b) 20-day average dollar volume $> \$20\text{M}$ (liquidity floor); (c) not a stablecoin/fiat pair; (d) not within 30 days of a known delisting. New listings wait 6 months before admission (listing-phase price action is its own chaotic regime).

39.2 Removal rules. Quarterly review removes pairs that: fall below the liquidity floor for 60+ days, get delisted, or develop data-quality issues that fail remediation. Removed pairs stay in the historical archive (their data remains valid for research) but leave the live universe.

39.3 Survivorship discipline. When researching cross-pair effects, include pairs that were delisted during the sample (pull them from the archive before removal). A universe of only today’s winners inflates every cross-sectional result.

39.4 Correlation management. Maintain a rolling 90-day BTC-correlation table for all pairs (auto-computed weekly). Uses: (a) the same-direction exposure limiter counts correlated longs as one position; (b) HRP clustering treats high-correlation pairs as one risk family; (c) when adding a new pair, prefer one whose strategy returns correlate < 0.5 with existing ones — diversification is measured on *strategy returns*, not tickers.

39.5 Universe tiers. T1 core (BTC, ETH — deepest liquidity, first to trade live), T2 majors (next 8 by volume), T3 satellites (rest — research only until Tier T2 capital). Live trading expands tier-by-tier only after paper results confirm per-tier slippage assumptions.

39.6 Pair-strategy fit matrix. Maintain a table of strategy $\times$ pair expectancy from the trade DB; review quarterly. Pairs where a strategy persistently fails get excluded from that strategy (documented as a filter), and the matrix often reveals structure (e.g., mean-reversion works on low-beta alts, momentum on high-beta) that feeds new miner filter ideas.

40. QA Quick-Reference Cards (Printable)

One card per gate — print these and check them physically. Full checklists with evidence requirements live in Part C; these are the at-a-desk summaries.

CARD G1 — Data Lake: all pairs queryable · spot-check 5 bars exact · gaps < 0.5% explained · resample test green · updater idempotent · views fast · disk logged

CARD G2 — Heartbeat: bars → features → sweep → trades in DB → slice with significance — all in one run, all logged, zero manual steps

CARD G3 — Mining: block catalog tested · 100k-variant run complete · every variant's trades in DB · plateau report · trial count recorded · data wall enforced

CARD G4 — Edge Analytics: fixture test · noise control ~0 cards · planted edge found · stability enforced · cards complete · expiry works

CARD G5 — Validation Lab: WF exact · MC calibrated · NOISE ≥95% KILLED · planted edge passes · PBO reference test · DSR trial-sensitive · verdicts persisted

CARD G6 — Risk & Portfolio: sizing hand-verified · all breakers drilled · VaR backtested · HRP constraints hold · weights sum to 1 · alerts received

CARD G7 — Command Center: 7 pages < 5s · numbers match SQL · no naked stats (n shown) · empty states safe · read-only except journal

CARD G8 — Paper Gate: 50+ trades · ≥3 strategies · drift ≤ 1.5σ · adherence ≥ 95% · zero breaker overrides · calibration ≤ 1.5×

CARD LIVE — Go-Live Gate: G8 passed · capital losable · one-page specs written · kill-switch drilled · backup restored · operator laws printed · breakers armed

CARD MONTHLY — Ops Health: rebalance decided · cards re-validated · miner re-run · slippage calibrated · breaker drill · backup verified · KPI board reviewed · plan updated

41. Case Studies — Anatomy of a Failure & of a Success

Two composite case studies showing the system thinking end-to-end. Expect your own versions of both within the first year.

41.1 Anatomy of a failure: “The Perfect Tuesday”

Discovery. A Stage-2 mining run surfaces T05+F05 (Tue) + X02 (2R) on BNBUSDT: expectancy +0.61R, n=88, PF 2.1. The Tuesday-only filter looks like magic.

Red flags the system raises automatically: 1. n=88 < 100 → provisional, can't be carded. Operator temptation: “close enough.” It is not. 2. The scanner tested 168 day×hour cells × 500+ combos — BH correction requires $p < 0.0012$; this slice sits at $p = 0.008$. **Fails FDR.** On noise alone we'd expect several cells this good. 3. Stability check: 100% of the edge comes from the 2021 bull half; the 2022 half is flat. Single-regime dependence. 4. Plateau: moving the day filter to Mon or Wed → negative. The “edge” is one cell wide — a spike, not a plateau.

Verdict: KILLED at Gate 1-adjacent checks; archived with reason MULTIPLE_TESTING + REGIME_FRAGILE. Cost of this discipline: nothing. Cost without it: ~3 months and a real-money drawdown discovering the same thing.

Lesson the database keeps: six months later, monthly re-mine finds the same Tuesday pattern again — now it's in the kill archive, cross-referenced, and auto-flagged. The system remembers so you don't re-pay tuition.

41.2 Anatomy of a success: "Boring Session Breakout"

Discovery. T09 session-breakout variant on ETHUSDT (the \$C2.4 walkthrough): +0.19R, n=631 — unimpressive next to the Tuesday miracle.

Why it survives: n large; plateau 0.875 (neighbors work); profitable in both halves; session-slice $p_{adj} = 0.003$ after BH; OOS +0.21R (expectancy went UP out-of-sample — happens; treat with same skepticism, verified no leak); WFE 0.71; MC dd_p95 13%; PBO 0.11; DSR $p=0.02$.

Life after validation: paper 41 trades +0.16R → live micro 60 trades +0.18R — decay z never below -1.2 — promoted to CORE at 100 live trades, sizing multiplier 1.25. Contributes a boring +1.2%/month at 0.75% risk.

The meta-lesson: the failure LOOKED better in every screenshot. The success looked boring in all of them. The system's gates exist because humans cannot tell these apart by looking — the failed one had the higher backtest expectancy by 3×. **This is the entire point of the architecture.**

42. The First 100 Live Trades — What to Expect

A statistical preview so nothing surprises you.

Trade 1–20: Everything feels significant. It isn't. n=20 has $SE \approx 0.27R$ — you could be +0.5R or -0.3R and both are consistent with your +0.2R backtest edge. Do not touch anything. Journal everything.

Trade 20–40: First real losing streak (60% chance of a 5+ streak by now — Appendix K.3). The decay detector may flicker to WATCH. This is normal; the detector is doing its job, and so are you by not panicking. Expect slippage slightly worse than modeled on ~20% of fills — feed it to calibration, not to emotion.

Trade 40–70: The first breaker event likely fires (daily -1.5% happens monthly on average). Follow PB-1. Notice how different real-money losses feel vs paper — that feeling is exactly why the operator laws exist. Emotion scores in your journal will drift up; the weekly review will catch it.

Trade 70–100: Drift z-score becomes meaningful. Most likely outcome: live expectancy 60–90% of backtest (costs, fills, your latency). That's success. If $\geq 60\%$: sizing law permits consideration of CORE promotion. If $< 40\%$: the strategy goes back through validation — evidence, not disappointment.

After trade 100: You now own something almost no retail trader has — a statistically meaningful live track record, fully journaled, with known slippage, known discipline metrics, and a validated (or correctly rejected) edge. Everything after this is scaling and repetition.

43. Document & Template Library

Standard documents to maintain in docs /. Templates below; keep them short and alive.

43.1 Module README template (one per src / module)

```
# <Module name>
Purpose: <one sentence>
Inputs: <data/contracts consumed>
Outputs: <data/contracts produced>
Config: <yaml keys used>
Owner: you | Status: build/testing/active
How to test: pytest tests/<module> – controls: <which statistical
controls apply>
Known limitations: <honest list>
```

43.2 Strategy one-pager template (required before live)

```
# <Strategy name> – <pair(s)> – <timeframe>
Logic in 60 seconds: <plain-language explanation>
Entry: <trigger> | Filters: <list> | Exit: <model>
Expected profile: WR ~x%, E[R]=y, n/month=z
Regime: thrives in <>, suffers in <>
Risk: stop <>, max concurrent <>, breaker links <>
Lineage: mined <run_id> → validated <run_id> → paper <dates> → live
<date>
Kill criteria: <what would make you retire it, written in advance>
```

43.3 Research log entry template (per mining/validation session)

```
Date: | Session goal: | Runs launched: <run_ids>
Found: <survivors> | Killed: <count + top reasons>
Surprises: <anything unexpected – these become hypotheses>
Next: <one action>
```

43.4 Decision log entry template

```
Date: | Decision: | Alternatives considered: | Why this one: | Revisit
when:
```

43.5 Incident report template (post-PB events)

```
Event: | Detected by: | Timeline: | Impact: | Root cause:
Fix applied: | Prevention added: | Affected runs flagged:
```

43.6 Monthly review minutes template

Month: | Equity: start/end | DD max: | MC percentile:
KPI reds: <list + action for each> | Strategies:
promoted/benched/retired
Rebalance: accepted/declined because | Calibration changes:
Discipline: adherence %, violations, theme | Next month's ONE
priority:

44. Year-2 Detailed Roadmap (Months 13–24)

Year 1 built the machine and proved the loop. Year 2 is about breadth, depth, and hardening — each quarter has one theme and explicit exit criteria.

Q1 (M13–15) — Theme: Breadth of edges

Workstream	Actions	Exit criteria
Miner-2 at full power	Weekly genetic runs; dedicated island-model tuning	≥ 2 GP-discovered strategies through the gauntlet
Forex plug-in live	Dukascopy lake complete; 4 forex strategies validated	Second market contributing live P&L
Funding-carry family	Delta-neutral funding harvest researched & validated	1 market-neutral strategy live (low correlation by construction)
Universe expansion	30 pairs total (tier rules \$39.5)	Liquidity floor maintained

Q1 risk focus: cross-market correlation audit — confirm crypto+forex portfolio DD is *lower* than crypto-only at same expectancy (it should be; if not, the allocation is wrong).

Q2 (M16–18) — Theme: Depth of research

Workstream	Actions	Exit criteria
Miner-3 ML filter deployed	Trade-success classifier as size adjuster on 3+ strategies	Measured expectancy lift vs control (paired test)
Order-book features	Snapshot collection running; imbalance blocks in catalog	≥ 1 book-feature filter survives validation
HMM regime overlay	Market-regime model gating strategy families	Overlay improves portfolio Sharpe in honest backtest
Failure research	Systematic study of the kill archive (all 12 months)	3 documented failure patterns → new miner constraints

Q3 (M19–21) — Theme: Operational hardening

Workstream	Actions	Exit criteria
Auto-execution graduation	If S1–S3 gates long since	2 months auto-exec matching

Workstream	Actions	Exit criteria
	green: Freqtrade deployment on 2 strategies	manual stats
UI v2 decision	Only if Streamlit genuinely limits daily ops	Decision logged with evidence
Tax & accounting pipeline	Journal → tax-lot export automation	One-click yearly report
Red-team week	Deliberately attack own system: inject bad data, fake edges, breaker storms	All defenses hold; gaps documented & fixed

Q4 (M22-24) — Theme: Scale & review

Workstream	Actions	Exit criteria
Capital tier T3 evaluation	24-month evidence review vs §30.1 law	Evidence-based tier decision
Portfolio-of-portfolios	Meta-allocation across families (trend/MR/carry/stat-arb)	Family-level HRP live
Annual deep review v2.0	Every strategy re-validated; this document revised	Master Plan v2.0 issued
Year-3 planning	Options? New venues? Larger infra? All gated by evidence	Written year-3 charter

Year-2 success definition (write it down now): a diversified, multi-market, multi-family portfolio with 200+ live trades of evidence, expectancy positive at portfolio level every rolling 6-month window, max DD inside MC bands, and an operator who followed the process ≥ 95% of days. If that sentence is true in month 24, you have built something genuinely rare.

45. The Daily Boot Sequence & Operating Manifesto

45.1 The 5-minute boot sequence (every trading day, in order)

1. System health: data fresh ✓ · jobs green ✓ · breakers armed ✓
[30 sec]
2. KPI reds: any red rows on the board? → resolve before cards
[60 sec]
3. Edge Cards: read today's cards aloud – conditions, size, stop
[2 min]
4. Risk budget: daily loss remaining, open exposure, VaR gauge
[60 sec]
5. Mindset: one breath, read laws #1 and #2, open journal
[30 sec]

If any step fails – no trading today, only fixing. A day without trades costs nothing; a day trading a broken system can cost the quarter.

45.2 The Operating Manifesto (print this page)

1. I trade the system's evidence, not my opinions.
2. I protect capital first; returns are the byproduct of survival.
3. I log everything, hide nothing, and review weekly without fail.
4. I let the validation lab break my heart cheaply so the market can't do it expensively.
5. I respect the sample size. I respect the correction. I respect the out-of-sample wall.
6. I size by formula, enter by card, exit by plan, and stop by breaker.
7. I expect losing streaks, drawdowns, dead months — and I have a written playbook for each.
8. I treat every kill as saved money and every mistake as tuition already paid.
9. I never scale on emotion — only on evidence, by the sizing law.
10. I am building a multi-year asset. I will not burn it for a one-week feeling.

Signed and dated, taped above the trading desk. Reread at every drawdown, every euphoria, and every temptation to “just this once.”

Appendix O — Pre-Live Simulation Drill Pack

A mandatory 2-week “dress rehearsal” between passing G8 and committing real capital. Treat it as seriously as flight-simulator hours.

Week 1 — Normal operations rehearsal - [] D1: Full boot sequence + 2 simulated trades end-to-end (alert → checklist → paper fill → journal) — timed; must feel boring - [] D2: Boot sequence + review yesterday's journal for completeness; fix form friction - [] D3: Simulate a 3-loss day in the journal; verify you followed law #4 (walk) and the daily digest still made sense - [] D4: Practice the monthly rebalance flow on last month's data (accept/decline with written reason) - [] D5: Simulate card expiry: pick an active card, walk through re-validation workflow

Week 2 — Emergency drills - [] D6: Breaker drill — force daily-loss breaker in a test account; confirm lockout + Telegram + your own compliance (no override) - [] D7: Kill-switch drill — flatten everything from your phone in < 60 seconds; log the time - [] D8: Outage drill — disconnect internet mid-session; execute PB-5 from written instructions only - [] D9: Data-corruption drill — quarantine a pair, flag affected runs, restore from backup (SYS-9 rehearsal) - [] D10: Full-stop rehearsal — walk PB-4 on paper: post-mortem template filled for a simulated -20% event

Graduation: all 10 drills logged with times and notes; any drill that felt confusing gets repeated until it doesn't. Capital is committed only after the drill pack is signed in the decision log.

Appendix P — Frequently-Used Reference Tables

P.1 z-score quick table (decay detector & drift tests)

z-score	Meaning	Action
> -1.0	Normal variance	Nothing
-1.0 to -1.5	Watch closely	Note in weekly review
-1.5 to -2.0	Warning	Size × 1 (watch state),

z-score	Meaning	Action
-2.0 to -2.5	Serious	investigate regime Size $\times$ 0.5 (automatic)
< -2.5 (20+ trades)	Likely edge failure	Bench $\rightarrow$ re-validation

P.2 Sample size cheat table ($\sigma(R) = 1.2$ assumed)

n trades	SE of mean R	Minimum E[R] distinguishable from 0 (95%)
25	0.24	0.47
50	0.17	0.33
100	0.12	0.24
200	0.085	0.17
400	0.06	0.12
1000	0.038	0.075

Use it inversely too: a +0.1R edge needs ~550+ trades to statistically confirm — some real edges are only provable across years or across pairs pooled.

P.3 p-value intuition table

Tests run	Naive $p < 0.05$ expected false hits	BH-adjusted threshold for top hit
1	0.05	0.050
20	~1	0.0025
168 (hour $\times$ day)	~8	0.0003
1,000	~50	0.00005

P.4 Win-rate / payoff break-even grid (before costs)

Win rate	Break-even payoff (avg win $\div$ avg loss)
35%	1.86
40%	1.50
45%	1.22
50%	1.00
55%	0.82
60%	0.67
65%	0.54

Trend systems live in the top rows (low WR, big payoff); mean-reversion in the bottom rows (high WR, small payoff). Know which row your strategy occupies — it predicts the losing-streak psychology you'll face.

P.5 Trade frequency planning table

Active strategies	Avg trades/strategy/month	Portfolio trades/month	Journal load
2	8	16	Light
4	8	32	Comfortable
6	10	60	Heavy — consider auto-journal
8	10	80	Maximum for manual ops

P.6 Expectancy health bands (live vs backtest)

Live ÷ backtest expectancy

	Reading	Action
> 100%	Suspiciously good	Verify no logging error; enjoy quietly
70–100%	Healthy	Nothing
50–70%	Acceptable	Check costs/slippage calibration
30–50%	Degraded	Reduce size; investigate fills
< 30%	Broken	Bench; re-validate

46. Master Index of Gates, Checklists & Controls

Every verification artifact in this document, indexed. When someone (including future-you) asks “how do you know X works?”, this index points to the proof.

46.1 Phase gates (Part E)

Gate	Verifies	Evidence location
G0	Repo, env, config, CI operational	\$24 Phase 0
G1	Data Lake complete & trusted	A1.1–A1.8
G2 ★	Heartbeat end-to-end loop	\$24 Phase 2
G3	Engine + Miner-1 at scale	A3.x, A4.x
G4	Trade DB + Edge Analytics honest	A5.x, A6.x
G5	Validation Lab rejects noise, keeps signal	A7.x

Gate	Verifies	Evidence location
G6	Portfolio + Risk enforced	A8.x, A9.x
G7	Command Center operable	A10.x
G8	Paper trading proven	\$24 Phase 8

46.2 Module acceptance checklists (Part C)

Checklist	Items	Module
A1.x	8	Data Lake
A2.x	7	Feature Factory
A3.x	8	Strategy Miner
A4.x	7	Backtest Engine
A5.x	7	Trade Database
A6.x	7	Edge Analytics
A7.x	7	Validation Lab
A8.x	5	Portfolio Engine
A9.x	6	Risk Engine
A10.x	6	Command Center
A11.x	6	Execution Loop
A12.x	5	Monitoring & Decay
Total	79 checks	—

46.3 Statistical controls (rerun after any statistics-code change)

Control	What it proves	Where
A6.3 noise scan	Edge engine produces ~0 false cards	\$14.5
A6.4 planted edge	Edge engine finds known edges	\$14.5
A7.3 noise gauntlet	Lab kills $\geq 95\%$ of random strategies	\$15.5
A7.4 planted strategy	Lab passes genuine persistent edges	\$15.5
A12.1 synthetic decay	Detector catches degrading stream	\$20.5
A2.1 lookahead suite	No future data in any feature	\$10.5

46.4 System-level acceptance tests

SYS-1 through SYS-10 in §26.2: rebuild-from-scratch, reproducibility, data-wall audit, cost realism, breaker drill, failure injection, performance, security, backup restore, documentation audit. All ten must pass before the first live trade — no exceptions, no “it’s probably fine.”

46.5 Operational drills (Appendix O)

D1–D10 dress-rehearsal drills: normal ops, breakers, kill-switch, outage, corruption, full-stop. Signed in the decision log before capital is committed.

46.6 Recurring verifications (never one-time)

Cadence	Verification
Daily	Boot sequence §45.1; data freshness; breaker status
Weekly	Journal audit; decay z-scores; backup ran
Monthly	Rebalance decision; card re-validation; slippage calibration; breaker drill; KPI board; restore spot-check
Quarterly	Universe review §39; failure-mode list re-read (App. H); correlation audit
Annually	Full re-validation of everything; Master Plan version bump

Appendix K — Compounding, Drawdown & Risk-of-Ruin Mathematics

The arithmetic of survival and growth — the tables to revisit before ever increasing size.

K.1 The compounding table (why small edges matter)

Monthly return vs equity growth on \$1,000 (no additions, no withdrawals):

Monthly	6 mo	12 mo	24 mo	36 mo
+1%	\$1,062	\$1,127	\$1,270	\$1,431
+2%	\$1,126	\$1,268	\$1,608	\$2,041
+3%	\$1,194	\$1,426	\$2,033	\$2,907
+5%	\$1,340	\$1,796	\$3,226	\$5,792
+8%	\$1,587	\$2,518	\$6,343	\$15,976

A +2–3% monthly system is unglamorous and excellent. Chasing +15% months is chasing variance that eventually collects its debt. Also note what the table demands: **not losing** — a –20% month wipes out ten +2% months.

K.2 The drawdown recovery trap

Drawdown	Gain needed to recover
-5%	+5.3%
-10%	+11.1%
-20%	+25%
-33%	+50%
-50%	+100%
-70%	+233%

This asymmetry is the mathematical reason the breaker stack exists. The -20% full-stop is placed exactly where recovery starts getting expensive; -50% is where systems go to die.

K.3 Consecutive losses are normal (psych-proof yourself)

With a 45% win-rate strategy, probability of a losing streak of length L somewhere in 100 trades:

Streak length	Probability in 100 trades
5 losses	~85%
7 losses	~45%
10 losses	~10%
12 losses	~3%

You WILL have an 8-loss streak while the system is perfectly healthy. The journal's streak profile (query C6) plus the decay z-score tell you when a streak stops being weather and starts being climate.

K.4 Risk-of-ruin intuition table

Approximate probability of hitting -20% before +40%, for $E[R]=+0.2$, $\sigma(R)=1.2$, various risk fractions (Monte Carlo, 50k paths):

Risk/trade	P(hit -20% first)
0.5%	~4%
0.75%	~6%
1.0%	~9%
1.5%	~16%
2.5%	~31%
5.0%	~55%

Read this table whenever greed whispers. Doubling from 1% to 2% doesn't double your expectation — it multiplies your ruin probability while expectancy grows only linearly.

K.5 The expectancy-frequency-size identity

Monthly return $\approx$ trades/month $\times$ E[R] $\times$ risk% $\times$ confidence
+2% month at 0.75% risk needs: 40 trades $\times$ +0.067R, or 12 trades $\times$ +0.22R,
or 5 trades $\times$ +0.53R

Three routes to the same destination: frequency (more strategies/pairs), edge (better filters), or size (more risk). Only the first two scale safely; the third is the ruin table above. This identity is why the system invests in mining breadth and filter quality rather than conviction sizing.

K.6 The cost drag table

Round-trip cost impact on a 30-trade/month strategy (0.75% risk, E[R]=+0.2 gross):

Round-trip cost	Cost/month	% of expectancy eaten
0.02R	0.6R	10%
0.06R	1.8R	30%
0.10R	3.0R	50%
0.15R	4.5R	75%

High-frequency small-edge strategies live and die on this table — the reason cost modeling, slippage calibration, and fee-tier management are engineering priorities, not afterthoughts.

Appendix L — Data Quality Field Guide

Symptoms you'll encounter in real data, what they mean, and what to do.

Symptom	Meaning	Action
Single 1m bar with 30% wick	Usually real (liquidation cascade); occasionally bad print	Check second source; if real, keep — your backtest MUST survive these
Multi-hour gap, all pairs	Exchange outage/maintenance	Document; exclude window from session stats
Gap on one pair only	Pair-specific halt or data issue	Quarantine, re-download
Volume = 0 for hours (alt, 2018)	Dead market period	Keep (it's history); liquidity floor filter handles it in strategies
Sudden price jump $\times 1000$ (old data)	Unit change/redenomination	Investigate pair history; adjust or exclude
Funding rate missing days	Perp not yet listed	Null, not zero — never fill with 0
Duplicate timestamps	Updater bug	A5.4 idempotency + dedupe migration
Negative volume	Corrupt row	Quarantine whole file, re-pull

Symptom	Meaning	Action
Timestamps off by exactly 1h	DST bug (forex later) or timezone mixup	Everything UTC, always; find the offending code
Latest bar stale > 24h	Updater dead	Health monitor should have paged you; check job log

Golden rules: (1) never silently “clean” data — quarantine, document, decide; (2) suspicious-but-real data stays in (markets are suspicious places); (3) every remediation is logged with date and reason; (4) re-run affected backtests after any historical correction and note metric deltas.

Appendix M — Builder’s Weekly Schedule Templates

Time is the scarcest input. Three templates for different life situations.

M.1 Full-time builder (25–30 h/week) — fastest path

Day	Block 1 (3h)	Block 2 (2h)
Mon	Module build (current phase)	Learning track
Tue	Module build	Tests + QA evidence
Wed	Module build	Learning track
Thu	Module build	Docs + decision log
Fri	Integration + fixes	Weekly review (runbook)
Sat	Optional: research notebooks	—
Sun	Rest	—

M.2 Part-time builder (10–12 h/week) — the default assumption

Day	1.5–2h block
Mon	Module build
Tue	Learning track + reading
Wed	Module build
Thu	Tests + QA evidence
Fri	Module build
Sun	Weekly review + planning (1h)

M.3 Operator phase (once live; 5–7 h/week)

Day	Time	Activity
Daily	20 min	Pre-session + journal (runbook §27.1)
Sun	1 h	Weekly review (§27.2)
1st Sun	3 h	Monthly deep-dive (§27.3)

Day	Time	Activity
Flexible	2–3 h	One improvement task from the monthly priorities — never more than one

Scheduling law: the system is built in the blocks, but it's *kept* in the reviews. If a week only allows one hour, spend it on the weekly review — continuity of observation beats bursts of effort.

Appendix N — Expanded Glossary of Quant Terms

Alpha decay — the tendency of a discovered edge to weaken as markets adapt. **Anchored walk-forward** — walk-forward where the training window's start stays fixed while it grows. **Autocorrelation** — correlation of a series with its own past; streaks in trade P&L. **Basis** — price difference between perp and spot. **Beta** — sensitivity to the market factor (BTC for alts). **Block bootstrap** — resampling contiguous chunks to preserve autocorrelation. **Breadth** — number of independent bets; Grinold's law: $\text{skill} \times \sqrt{\text{breadth}}$. **Calmar ratio** — $\text{CAGR} \div \text{max drawdown}$. **Chandelier exit** — ATR-based trailing stop hanging from the highest high. **Cointegration** — long-run statistical linkage between non-stationary series (pairs trading basis). **Cross-sectional** — comparing many assets at one time (vs time-series: one asset across time). **CSCV** — combinatorially symmetric cross-validation; the PBO machinery. **CVaR / Expected Shortfall** — average loss in the worst $\alpha\%$ of cases. **Deflated Sharpe** — Sharpe significance deflated by trial count and non-normality. **Drawdown duration** — time spent below the equity peak; hurts more than depth. **Edge** — positive expectancy that persists out-of-sample and net of costs. **Embargo** — gap enforced between train and test windows to prevent leakage. **Expectancy** — mean P&L per trade (R or bps). **Exposure** — fraction of time/capital in the market. **Fat tails** — extreme events more frequent than Gaussian prediction. **Fill model** — assumptions about how orders execute in backtest. **Funding rate** — perp mechanism payment anchoring perp to spot. **GARCH** — volatility clustering model; forecasts tomorrow's vol. **HRP** — clustering-based allocation needing no covariance inversion. **Information Coefficient** — rank correlation of signal vs forward return. **Intrabar ambiguity** — unknowable order of high/low within one bar. **Kelly criterion** — growth-optimal bet fraction; used fractionally if at all. **Kupiec test** — statistical check of VaR model exception rate. **Leakage** — any future information contaminating the past; the original sin. **MAE/MFE** — worst/best excursion during a trade's life. **Multiple comparisons problem** — more tests $\Rightarrow$ more false discoveries. **Overfitting** — fitting noise; performs in-sample, dies live. **PBO** — probability the selected config is overfit. **Plateau** — parameter region where neighbors also perform. **Point-in-time** — data as known at that moment, no revisions. **Profit factor** — $\text{gross profit} \div \text{gross loss}$. **Purged CV** — cross-validation removing overlapping label windows. **R-multiple** — P&L in units of initial risk. **Regime** — persistent market state with stable statistics. **Risk parity** — allocating so each component contributes equal risk. **Sharpe ratio** — excess return per unit volatility. **Slippage** — execution price worse than decision price. **Sortino** — Sharpe using downside deviation only. **Stationarity** — stable statistical properties over time. **Stochastic dominance** — one distribution systematically better than another. **Survivorship bias** — measuring only what survived. **Tail risk** — exposure to extreme, rare losses. **Time-stop** — exit after fixed holding period. **Turnover** — trading frequency; drives cost drag. **Type I/II error** — trading noise / discarding signal. **VaR** — loss level not exceeded

with probability α . **Volatility targeting** — scaling size to hold portfolio vol constant. **Walk-forward efficiency** — $\text{OOS} \div \text{IS}$ annualized performance. **Wick** — bar extreme beyond body; rejection evidence. **z-score** — distance from mean in standard errors.

Final word. This plan is deliberately conservative: it assumes most ideas fail, that noise impersonates edge, and that the operator is human. Systems built on those assumptions are the ones still trading in year three. Build the heartbeat first (G2), protect the database like treasure, let the Validation Lab break your heart weekly — and let compounding do what compounding does.

— *End of Master Plan v1.0* —